SUMMARIV: A MULTIMODAL RETRIEVAL AUGMENTED FRAMEWORK FOR EVIDENCE GROUNDED VIDEO ANSWERING AND VIDEO SUMMARIZATION

Project Report Submitted in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

by

T. Tej Mahendra Reddy (22501A05H9) **T. Vimala Reddy (22501A05I4)**

P. Kamal Siddardha (22501A05D5) **Ruksana Shaik (22501A05F6)**

Under the guidance of

Ms. Y. Manasa, MTech., (Ph.D.)
Assistant Professor

Department of Computer Science and Engineering

Prasad V Potluri Siddhartha Institute of Technology

(Permanently affiliated to JNTU-Kakinada, Approved by AICTE)

(An NBA & NAAC A+ accredited and ISO 21001:2018 certified institute)

Kanuru, Vijayawada-520 007

2025-26

Department of Computer Science and Engineering

Prasad V Potluri Siddhartha Institute of Technology

(Permanently affiliated to JNTU-Kakinada, Approved by AICTE)

(An NBA & NAAC A+ accredited and ISO 21001:2018 certified institute)

Kanuru, Vijayawada-520 007

CERTIFICATE

This is to certify that the project report entitled **"SUMMARIV: A Multimodal Retrieval-Augmented Framework for Evidence-Grounded video Answering and Video Summarization"** that is being submitted by Tatiparthi Tej Mahendra Reddy (22501A05H9), Tummuru Vimala Reddy (22501A05I4), Parsa Kamal Siddardha (22501A05D5), Ruksana Shaik (22501A05F6), in partial fulfillment for the award of the Degree of Bachelor of Technology in Computer Science and Engineering during the academic year 2025-26.

Signature of the guide
Ms. Y. Manasa
Assistant Professor

Signature of the HOD
Dr. A. JayaLakshmi
Professor and HOD

SIGNATURE OF THE EXTERNAL EXAMINER

DECLARATION

We declare that project work entitled "**SUMMARIV: A Multimodal Retrieval-Augmented Framework for Evidence-Grounded video Answering and Video Summarization**" is composed by ourselves, that the work contained herein is our own except where explicitly stated otherwise in the text, and that this work has not been submitted for any other degree or professional qualification except as specified.

Tatiparthi Tej Mahendra Reddy (22501A05H9)

Tummuru Vimala Reddy (22501A05I4)

Parsa Kamal Siddardha (22501A05D5)

Ruksana Shaik (22501A05F6)

Acknowledgement

We sincerely thank the following distinguished personalities who have given their advice and support for successful completion of the work.

We are deeply indebted to our most respected guide **Ms. Y. Manasa, Assistant Professor,** Department of computer science and engineering for her valuable and inspiring guidance, comments, suggestions and encouragement.

We extend our sincere thanks to **Dr. A. Jayalakshmi, Professor & Head of the department** for extending her cooperation and providing the required resources.

We would like to thank our beloved **Principal, Dr. K. Sivaji Babu** for providing the online resources and other facilities to carry out this work.

We sincerely express our gratitude to **Dr. Ravuri Daniel, Associate Professo**r, our Overall Project Coordinator of the department, for his support throughout the completion of this work.

We're grateful to **Mrs. A. Divya, Assistant Professor** of our department and our esteemed Project Coordinator for her invaluable guidance and efforts. Her exceptional leadership, from inception to final review, ensured the project's success.

We would like to extend our sincere gratitude to all the review panel **Dr. P. Sai Kiran, Professor, Dr. S. Phani Praveen, Associate Professor and Dr. J. Rama Devi, Sr. Assistant Professor,** for their valuable suggestions in project reviews**.**

We extend our sincere thanks to all other teaching and non-teaching staff of the department, for their cooperation and encouragement.

Tatiparthi Tej Mahendra Reddy (22501A05H9)

Tummuru Vimala Reddy (22501A05I4)

Parsa Kamal Siddardha (22501A05D5)

Ruksana Shaik (22501A05F6)

ABSTRACT

The rapid growth of video-based content across educational, informational, and multimedia platforms has created a significant demand for intelligent systems capable of efficiently understanding and retrieving relevant information from videos. Traditional video analysis methods often rely on manual viewing or basic metadata, which can be time-consuming and insufficient for deep content understanding. Moreover, extracting meaningful insights from both audio and visual components of a video remains a complex challenge due to the multimodal nature of the data.

To address these challenges, a Multimodal Video Understanding and Question Answering Framework is proposed. The system integrates advanced deep learning and natural language processing techniques to analyse both speech and visual content within a video. Audio information is processed using automatic speech recognition to generate time-aligned transcripts, while visual information is extracted through frame sampling and image captioning models to capture scene-level context. These multimodal data representations are then transformed into vector embeddings and indexed using an efficient similarity search mechanism.

The framework employs a Retrieval-Augmented Generation (RAG) approach, where relevant audio and visual contexts are retrieved based on user queries and passed to a lightweight language model to generate precise answers. Additionally, the system enhances user experience by extracting and presenting the exact video segment corresponding to the retrieved answer, enabling quick and intuitive understanding.

By combining multimodal learning, semantic search, and generative modelling, the proposed system improves the accuracy, efficiency, and interpretability of video content analysis. This approach contributes to advancements in intelligent video retrieval, educational assistance, and interactive multimedia systems, enabling users to access relevant information quickly and effectively from large video datasets.

LIST OF FIGURES

FIG. NO	FIGURE NAME	PG. NO
5.1.1	Architecture of Proposed System	17
5.2.1	Detailed System Design of SummariV Framework	21
5.2.2	Proposed System Flow Diagram	23
5.2.3	Sequence Diagram	24
8.1	Grouped Bar Chart — AR, EGS, and TLA Across All Eight Configurations	44
8.2	Heatmap — AR, EGS, and TLA Across All Configurations	44
8.3	Performance Comparison Across Methods and Configurations	46
8.4	Progressive Performance Graph from LLM-only to Full Configuration	46
8.5	Component Ablation Study	47
8.6	Metric Coverage — Base vs Full-Large Configuration	48
8.7	Score Distribution Across All Evaluation Queries	49
A.1	Video upload and pipeline interface	69
A.2	Pipeline processing stages	69
A.3	Completed indexing and system ready state	69
A.4	Evidence-based answer with video clip	70
A.5	Overall video summary output	70
A.6	Query handling with low confidence response	70

LIST OF TABLES

FIG. NO	TABLE NAME	PG. NO
7.2.1	Test Cases	40
8.1	Parameter Configurations	41
8.2	Overall Performance Across Configurations	43
8.3	Comparison with Baseline Approaches and Progressive Component Upgrades	45
8.4	Component ablation study	47
B.1	Mapping of SDGs	71

TABLE OF CONTENTS

NAME	Pg No
CHAPTER 1: INTRODUCTION	**1**
1.1 Introduction	1
1.2 Motivation	4
1.3 Statement of the problem and Solution to Problem	4
1.3.1 Problem statement	
1.3.2 Solution	
1.4 Objectives	5
1.4.1 General Objective	
1.4.2 Specific Objectives	
1.5 Scope of the Work	6
1.6 Significance of the Work	6
1.7 Outlines of the Project	7
CHAPTER 2: BACKGROUND AND LITERATURE REVIEW	**9**
CHAPTER 3: SYSTEM ANALYSIS	**12**
3.1 Existing System	12
3.1.1 Traditional Clinical Assessment	
3.1.2 Rule-based and Statistical Approaches	
3.1.3 Machine Learning-Based Models	
3.2 Proposed System	13
3.3 Feasibility Study	13
3.3.1 Technical Feasibility	
3.3.2 Operational Feasibility	
3.3.3 Economic Feasibility	
CHAPTER 4: SOFTWARE REQUIREMENT SPECIFICATION	**15**
4.1 Functional Requirements	15
4.2 Non-Functional Requirements	16

CHAPTER 5: DESIGN AND METHODOLOGY OF PROPOSED SYSTEM **17**
5.1 System architecture 17
5.2 System Design 20
5.2.1 UML Diagrams
5.3 Methodology 25
5.3.1 Data Collection
5.3.2 Process Model
5.3.3 Tools and Techniques

CHAPTER 6: IMPLEMENTATION **30**
6.1 Modules 30
6.2 Description of Sample code of Each Module 32

CHAPTER 7: TESTING **37**
7.1 Testing Strategy 37
7.2 Test Cases 40

CHAPTER 8: RESULT AND DISCUSSION **41**

CHAPTER 9: CONCLUSION AND FUTURE WORK **50**
9.1 Conclusion 50
9.2 Future Work 50
REFERENCES & BIBLIOGRAPHY 51
APPENDIX A 53
Full Code 53
Screenshots 69
APPENDIX B 71
Mapping of Sustainable Development Goals 71

CHAPTER - 1
INTRODUCTION

1.1 Introduction

The exponential proliferation of digital multimedia content, particularly long-form educational video recordings, has fundamentally transformed the landscape of knowledge dissemination in the modern era. Online learning platforms such as YouTube, Coursera, edX, and Khan Academy collectively host hundreds of millions of hours of instructional video content spanning virtually every academic discipline and professional domain. While this unprecedented democratization of educational resources offers immense societal benefit, it simultaneously introduces a critical usability challenge that has remained largely unaddressed: learners who seek a specific piece of information embedded within a lengthy lecture or tutorial recording are compelled to watch the entire video from start to finish, or at best, manually seek through the recording without any intelligent guidance. This process is cognitively demanding, time-consuming, and fundamentally inefficient in an era where information retrieval is expected to be instantaneous and precise.

The challenge is further compounded by the inherently multimodal nature of educational video content. In a typical academic lecture, an instructor simultaneously speaks about a concept verbally while displaying supporting visual aids such as slides, diagrams, mathematical equations, flowcharts, and on-screen demonstrations [1]. These two channels of information, the spoken audio and the displayed visual content, are complementary and together constitute the complete informational content of the lecture. Systems that process only the audio transcript of a video therefore capture only a partial representation of the knowledge conveyed, missing the substantial portion communicated through visual means. For example, a lecturer explaining the architecture of a neural network will typically refer to a diagram on a slide while speaking, and a transcript-only system that captures only the spoken words loses the critical contextual information provided by the diagram itself.

Existing approaches to navigating and querying video content fall broadly into three categories, each with significant limitations. The first category comprises generic video summarization systems [2,3], which generate a fixed, query-independent textual summary of the video's content. While such summaries are useful for obtaining a general overview of a video before deciding whether to watch it, they are fundamentally incapable of addressing a specific, targeted question that a learner might have about the video. The second category

consists of Large Language Model (LLM)-based question answering systems [4,5], which accept a user's natural language question and generate a fluent response. However, because these systems generate answers based entirely on their parametric training knowledge rather than the actual content of the video being queried, their responses are frequently hallucinated, meaning they are plausible sounding but factually incorrect or unverifiable [6] against the actual video content. This limitation is particularly serious in educational settings where accuracy is paramount. The third category comprises transcript-based retrieval systems [7,8], which transcribe the video's audio and allow users to search for keywords within the transcript. While this approach provides some grounding in the video's actual content, it relies exclusively on the audio modality [9], misses all visual information, and typically does not provide precise temporal localization of where within the video the relevant information appears.

In response to these limitations, this project presents SummariV, a novel Multimodal Retrieval-Augmented Generation (RAG) framework [10] specifically designed for evidence-grounded video question answering and summarization of long-form educational video content. The name SummariV reflects its dual purpose: providing targeted summaries of video content in response to specific user queries while simultaneously grounding every generated response in verifiable, timestamped video evidence.

The SummariV system processes an input educational video through two parallel content extraction pipelines that operate simultaneously to capture the complete informational content of the video across both its modalities. The first is a Speech-Aware Content Extraction Pipeline, which employs OpenAI Whisper [11], a state-of-the-art automatic speech recognition model, to transcribe the spoken audio content of the video into a series of timestamped text segments. Each segment captures a sentence or phrase of spoken content along with its precise start and end timestamps within the video. The second is a Vision-Based Frame Extraction and Captioning Pipeline, which uses OpenCV to extract representative video frames at fixed two-second intervals throughout the video's duration, and then applies the Salesforce BLIP (Bootstrapped Language-Image Pretraining) vision-language model [12] to generate natural language captions describing the visual content of each extracted frame. Each caption is associated with the timestamp of its corresponding frame.

Once both pipelines have processed the video, all generated text segments and visual

captions are encoded into dense 384-dimensional semantic vector representations using the Sentence-BERT [13] model with the all-MiniLM-L6-v2 architecture. These vector representations capture the deep semantic meaning of each content unit in a form that enables efficient mathematical comparison. The embeddings from both modalities are stored in two separate FAISS (Facebook AI Similarity Search) [14] vector indexes, one for audio-derived transcript segments and one for vision-derived captions, which support highly efficient similarity-based retrieval at query time.

When a user submits a natural language question about the video, the system encodes the query using the same Sentence-BERT model and performs cosine similarity search against both FAISS indexes simultaneously to retrieve the top-k most semantically relevant evidence units from each modality. The retrieved audio and visual evidence segments are concatenated into a combined multimodal context, which is passed to FLAN-T5 [15], a lightweight instruction-following language model, with an explicit constraint that the generated answer must be derived solely from the provided context. This evidence-constrained generation mechanism [16] fundamentally prevents the hallucination that characterizes LLM-only approaches, because the model cannot generate information that is not present in the retrieved video evidence. Finally, the system uses FFmpeg to extract and present the precise video clip corresponding to the temporal location of the best-matching evidence segment, enabling the user to immediately view the relevant portion of the video in context.

The SummariV system is evaluated on educational lecture videos from the publicly available EDU-VSUM [17] dataset using three evaluation metrics proposed specifically for this system: Answer Relevance (AR), which measures whether the generated answer correctly addresses the user's query; Evidence Grounding Score (EGS), which measures the extent to which the generated answer is explicitly supported by the retrieved video evidence; and Temporal Localization Accuracy (TLA), which measures whether the extracted video clip correctly corresponds to the temporal segment containing the answer. Comparative evaluation against an LLM-only baseline and a transcript-only retrieval baseline demonstrates that SummariV achieves superior performance across all three metrics, with AR of 0.834, EGS of 0.753, and TLA of 0.739. Most significantly, SummariV reduces the average video content that a user must access to find an answer from four to six minutes of full video viewing down to five to eight seconds of extracted clip, representing a reduction of over 98% in information access time.

The system is implemented as a complete, executable pipeline in the Google Colab environment using Python and a suite of open-source deep learning libraries including PyTorch, HuggingFace Transformers, FAISS, OpenCV, and FFmpeg, making it accessible to researchers and educators without requiring specialized hardware infrastructure beyond standard GPU computing resources.

1.2 Motivation

The rapid growth of video-based content across educational, professional, and digital platforms has created a strong need for intelligent systems that can efficiently understand and retrieve relevant information from videos. Traditional approaches require users to manually watch entire videos, making information access time-consuming and inefficient. Moreover, the inability to effectively combine audio and visual data limits accurate content understanding, highlighting the need for advanced automated solutions.

Recent advancements in artificial intelligence, particularly in natural language processing and computer vision, provide new opportunities to transform video analysis. By leveraging multimodal data such as speech transcripts and visual cues, the proposed framework enables automated content understanding, semantic search, and question answering. The motivation behind this project is to enhance accessibility, reduce information retrieval time, and support users with precise, context-aware insights, ultimately contributing to smarter and more efficient multimedia interaction systems.

1.3 Statement of the problem and Solution to Problem

1.3.1 Problem statement

The rapid increase in video-based content across various platforms has created a significant challenge in efficiently accessing and understanding relevant information. Traditional video consumption methods require users to manually watch entire videos, which is time-consuming and inefficient, particularly for lengthy educational or informational content. Existing systems often rely on basic metadata or keyword-based search, which fails to capture the deeper semantic meaning of video content. Additionally, the inability to effectively integrate both audio and visual information limits accurate content retrieval and understanding. Therefore, there is a need for an intelligent system that can automatically analyze, index, and retrieve meaningful information from videos using advanced data-driven techniques.

1.3.2 Solution

To address these challenges, a Multimodal Video Understanding and Question Answering Framework is proposed. This system leverages advanced artificial intelligence techniques, including speech recognition, image captioning, and semantic search, to process both audio and visual components of a video. By converting video content into structured textual and visual representations and storing them as vector embeddings, the system enables efficient retrieval of relevant information. The integration of a retrieval-augmented generation approach allows users to ask natural language queries and receive precise, context-aware answers. Additionally, the system enhances usability by providing the exact video segment corresponding to the answer, improving interpretability and user experience. This approach significantly improves the efficiency, accuracy, and accessibility of video content analysis, contributing to smarter and more interactive multimedia systems.

1.4 Objectives

1.4.1 General Objective

- Improve accessibility of video content by enabling intelligent understanding of multimedia data
- Enhance efficiency of information retrieval from large video datasets
- Integrate audio and visual intelligence using advanced artificial intelligence techniques
- Design and implement a comprehensive system for automatic video analysis and understanding
- Utilize speech recognition, computer vision, and natural language processing for extracting meaningful insights
- Enable users to quickly access relevant information through an intelligent multimodal framework
- Support interactive and user-friendly exploration of video content

1.4.2 Specific Objectives

- Develop a multimodal framework that processes both audio and visual components of video content
- Generate accurate speech transcripts using speech-to-text models
- Extract visual context from video frames using image captioning techniques

- Convert multimodal data into vector embeddings for efficient semantic search
- Enable natural language-based question answering over video content
- Retrieve relevant context using similarity-based retrieval methods
- Generate precise and context-aware responses using retrieval-augmented generation
- Integrate video clip extraction for temporal localization of answers
- Improve answer accuracy while reducing information retrieval time
- Provide an intuitive and interactive system for understanding and exploring video content

1.5 Scope of the Work

The proposed solution, the Multimodal Video Understanding and Question Answering Framework, addresses the growing need for efficient video content analysis and retrieval. Its scope focuses on enabling automated understanding of both audio and visual components of videos, improving information accessibility, and reducing the time required for manual video exploration. The framework aims to streamline video processing by generating transcripts, extracting visual context, and organizing data using semantic embeddings for effective search and retrieval. Additionally, it supports interactive querying and precise answer generation, making video content more accessible and user-friendly across educational and informational domains.

1.6 Significance of the Work

The development of a Multimodal Video Understanding and Question Answering Framework represents the integration of advanced technologies, including artificial intelligence, natural language processing, and computer vision. By applying these technologies to video analysis, the project contributes to advancements in intelligent multimedia systems and establishes a foundation for future innovations in automated content understanding and retrieval.

The significance of this work extends beyond technical advancement to practical applications in education, research, and digital content management. By enabling users to quickly access relevant information and visualize exact video segments, the system enhances learning efficiency, improves user experience, and supports effective knowledge discovery. This project promotes smarter interaction with video data and contributes toward building more accessible, efficient, and intelligent multimedia solutions for diverse user needs.

1.7 Outlines of the Project

1.7.1 Introduction

- Overview of the project's objectives and significance.
- Brief explanation of the multi-model framework and its role in video understanding and question answering.

1.7.2 Literature Review

- Review of existing research and technologies related to video analysis, speech recognition, and visual understanding.
- Discussion of previous works utilizing machine learning, natural language processing, and computer vision for video content retrieval and analysis.

1.7.3 Methodology

- Dataset description covering video input collection, preprocessing, and extraction of both audio and visual data.
- Development of a multimodal framework integrating speech-to-text models, image captioning techniques, embedding generation, and semantic search mechanisms.
- Design of a user interaction system enabling efficient querying of video content and retrieval of relevant, evidence-based answers.

1.7.4 System Architecture

- Overview of the system architecture highlighting key components and their interactions within the pipeline.
- Explanation of how audio and visual data are processed and integrated within the multimodal framework.
- Description of backend infrastructure, including data storage, embedding indexing, and processing mechanisms.

1.7.5 Implementation

- Technical details of the implementation including tools, frameworks, and libraries utilized in system development.
- Challenges encountered during development and the solutions implemented.

- Discussion of system deployment and scalability considerations.

1.7.6 Results

- Evaluation of system performance based on retrieval accuracy and answer relevance.
- Presentation of experimental results demonstrating the effectiveness of multimodal understanding.
- Comparison of system performance with traditional video search and retrieval methods.

1.7.7 Conclusion

- Summary of key findings and system capabilities.
- Reiteration of the significance of the project in improving video content accessibility and interaction.
- Future scope for enhancements, including improved models and real-time processing capabilities.

1.7.8 References

- Citations of relevant research papers, tools, and technologies used during the project development.

CHAPTER - 2
BACKGROUND AND LITERATURE REVIEW

Karpathy and Fei-Fei (2015), [1], introduced deep visual-semantic alignment techniques to map images and text into a shared embedding space. This approach enables better understanding of relationships between visual and textual data. It forms the foundation for multimodal retrieval systems. It is widely used in image captioning and video understanding.

Zhou et al. (2018), [2], presented a comprehensive survey on deep learning approaches for video summarization. The study categorizes techniques into traditional and deep learning-based methods. It highlights the effectiveness of extracting key frames and segments. However, most methods are query-independent and lack user-specific relevance.

Zhang et al. (2021), [3], proposed an LSTM-based approach for video summarization, focusing on capturing temporal dependencies in video sequences. The model identifies important segments based on sequential patterns. While effective, it suffers from limitations in handling long-range dependencies. Transformer-based methods later improved these shortcomings.

Brown et al. (2020), [4], presented GPT-3, a large-scale language model capable of performing multiple tasks with minimal examples through few-shot learning. The model demonstrated strong capabilities in text generation, reasoning, and conversational AI. However, it lacks grounding in external data, often leading to hallucinated responses. This limitation motivates the need for retrieval-based approaches.

Radford et al. (2019), [5], developed GPT-2, demonstrating that large language models can perform multiple NLP tasks without explicit supervision. The model uses unsupervised learning on large datasets to generate coherent text. It highlights the scalability of Transformer-based architectures. However, it lacks mechanisms for grounding outputs in evidence.

Chen et al. (2023), [6], addressed hallucination issues in large language models by introducing grounding techniques. The study emphasizes generating responses based on verifiable evidence. It improves the reliability and trustworthiness of AI systems. This motivates the use of retrieval-augmented frameworks.

Devlin et al. (2019), [7], proposed BERT, a bidirectional Transformer-based model designed to understand contextual relationships in text. Unlike previous models, BERT processes input in both directions, improving performance in tasks such as question

answering and sentence similarity. It introduced pre-training and fine-tuning strategies that became standard in NLP. BERT plays a key role in semantic embedding and retrieval tasks.

Lin et al. (2019), [8], extended BERT for video-language understanding by integrating visual and textual features. The model improves performance in tasks such as video retrieval and captioning. It enables better alignment between video frames and language representations. However, it does not address query-aware retrieval or temporal grounding.

Baevski et al. (2020), [9], introduced wav2vec 2.0, a self-supervised model for speech representation learning. It converts raw audio into meaningful embeddings for speech recognition tasks. The model significantly improves transcription accuracy. It plays a crucial role in extracting speech content from videos.

Lewis et al. (2020), [10], introduced Retrieval-Augmented Generation (RAG), which combines information retrieval with language generation. The model retrieves relevant documents and uses them as context for generating responses, improving factual accuracy. This approach reduces hallucinations and enhances reliability.

Radford et al. (2023), [11], introduced Whisper, a robust automatic speech recognition (ASR) model designed to generalise across diverse acoustic conditions, including noise, accents, and varying recording quality. The model outputs time-aligned word-level transcripts, which enables precise temporal mapping of spoken content to video segments.

Li et al. (2022), [12], proposed BLIP (Bootstrapped Language-Image Pre-training), a unified vision-language framework for image-text understanding and generation. The model supports conditional image captioning, visual question answering, and image-text retrieval within a single architecture.

Reimers and Gurevych (2019), [13], proposed Sentence-BERT, which improves sentence embeddings using Siamese networks. It enables efficient computation of semantic similarity between sentences. The model is widely used in information retrieval and clustering tasks. It plays a key role in similarity-based retrieval systems.

Johnson et al. (2021), [14], developed FAISS (Facebook AI Similarity Search), a library for efficient similarity search and clustering of dense vectors. FAISS is specifically designed to handle billion-scale vector databases through optimised indexing structures including IndexFlatL2, IVF, and HNSW.

Raffel et al. (2020), [15], proposed the T5 model, which unifies all NLP tasks into a text-to-text framework. The model simplifies task design and improves transfer learning

performance. It can perform tasks such as summarization, translation, and question answering. T5 is widely used for generating structured responses.

Wu et al. (2022), [16], introduced evidence-aware retrieval methods to ensure that generated answers are grounded in retrieved data. The approach reduces hallucinations by constraining outputs to verified evidence. It improves reliability in question answering systems.

Zhang et al. (2023), [17], proposed the EDU-VSUM dataset focused on educational video summarization. It contains structured lecture-style videos with both speech and visual content. The dataset is suitable for evaluating query-aware and multimodal systems.

Jang et al. (2020), [18], proposed a video question answering framework using spatio-temporal reasoning. The model captures relationships across video frames and time to improve answer accuracy. While effective, it is computationally expensive and lacks strong grounding in evidence. It also struggles with long-form video understanding.

Tan and Bansal (2020), [19], introduced a multimodal Transformer model for video retrieval, enabling joint representation learning of video and text. The model improves semantic alignment across modalities. It enhances retrieval performance in multimodal tasks. However, it lacks query-specific evidence grounding and temporal localization.

Li et al. (2022), [20], proposed VideoLLM, which integrates large language models with video understanding. The model enables reasoning over video sequences using multimodal inputs. While powerful, it is computationally expensive and often lacks precise temporal grounding. It may also generate unverified outputs.

From the reviewed literature, it is evident that significant advancements have been made in transformer-based architectures, multimodal learning, and video understanding techniques. However, most existing approaches either focus on query-independent summarization, lack evidence grounding, or fail to provide precise temporal localization of relevant video segments. These limitations highlight the need for a unified framework which integrates multimodal data, retrieval-augmented generation, and evidence grounding to provide accurate, interpretable, and query-aware video summarization and question answering.

CHAPTER – 3
SYSTEM ANALYSIS

3.1 Existing System

3.1.1 Traditional Video Consumption

Traditional methods of understanding video content primarily rely on manual viewing, where users watch entire videos to extract relevant information. This process is time-consuming, inefficient, and impractical for large or lengthy video datasets. Additionally, users often struggle to locate specific segments of interest without proper indexing or navigation mechanisms.

3.1.2 Metadata and Keyword-Based Approaches

Conventional systems utilize metadata such as titles, tags, and descriptions, along with keyword-based search techniques, to retrieve video content. These approaches depend heavily on manually provided information and exact keyword matching, which limits their ability to capture the semantic meaning of video content. As a result, retrieval accuracy is often low, and relevant information may be overlooked.

3.1.3 Machine Learning-Based Models

- Machine learning models have been applied to video analysis tasks such as speech recognition, object detection, and content classification. These systems process either audio or visual data independently to extract useful information.
- Such models involve stages including data preprocessing, feature extraction, model training, and evaluation. For example, speech-to-text models convert audio into text, while computer vision models analyze visual frames for object or scene recognition.
- However, traditional approaches often focus on single-modality inputs, either audio or visual, limiting their ability to fully understand video content. These systems may require large datasets and computational resources, and the lack of integration between modalities reduces overall efficiency and contextual understanding.

3.2 Proposed System

- The proposed system is a Multimodal Video Understanding and Question Answering Framework designed to analyze and retrieve meaningful information from video content using both audio and visual data.
- The fundamental idea behind the system is to combine speech recognition and image captioning techniques to extract structured information from videos, enabling efficient semantic understanding and retrieval.
- The framework processes audio by generating time-aligned transcripts and processes visual data by extracting frames and generating descriptive captions, which together provide comprehensive context.
- The system architecture includes preprocessing stages, embedding generation, and indexing using vector databases, allowing efficient similarity-based search and retrieval of relevant information.
- A retrieval-augmented generation approach is used, where relevant audio and visual contexts are retrieved based on user queries and passed to a language model to generate accurate and concise answers.
- The system also includes a video clip extraction module that identifies and displays the exact segment corresponding to the retrieved answer, enhancing interpretability and user experience.

3.3 Feasibility Study

3.3.1 Technical Feasibility

Technical feasibility depends on the availability of video data and pre-trained models capable of processing both audio and visual information. The system utilizes well-established deep learning models for speech recognition, image captioning, and natural language processing, reducing the need for extensive training from scratch. Efficient handling of video data requires sufficient computational resources, including GPU support for faster processing and optimized libraries for handling large-scale data.

The implementation leverages existing frameworks and tools to ensure reliable performance while minimizing development complexity. Techniques such as embedding optimization and efficient indexing are used to reduce retrieval time without compromising accuracy. The use of modular components ensures flexibility and

adaptability to different types of video data.

3.3.2 Operational Feasibility

Operational feasibility is based on the system's ability to effectively process video content and provide accurate, real-time responses to user queries. The framework is designed to support interactive querying, enabling users to quickly retrieve relevant information without manually watching entire videos. Low-latency retrieval mechanisms ensure that answers are generated efficiently, improving user experience.

The system is scalable and capable of handling varying volumes of video data. It can be extended to support multiple users and large datasets, making it suitable for applications in education, research, and content management. Additionally, the framework can be integrated with existing platforms to enhance video search and analysis capabilities.

3.3.3 Economic Feasibility

The proposed system reduces the time and effort required for manual video analysis, leading to increased productivity and efficiency. By automating content understanding and retrieval, it minimizes the need for human intervention, thereby reducing operational costs in the long run.

Although initial setup may require computational resources and infrastructure, the use of open-source models and frameworks helps in minimizing development costs. Over time, the system provides significant value by improving information accessibility, supporting learning processes, and enabling efficient content utilization, making it a cost-effective solution for multimedia analysis.

CHAPTER - 4

SOFTWARE REQUIREMENT SPECIFICATION

4.1 Functional Requirements

1. **Operating System:** Windows or any compatible operating system (Linux / MacOS supported).
2. **Processor:** Minimum 1 GHz processor or higher
3. **RAM:** 8 GB and above
4. **Hard Disk:** 4 GB and above
5. **Programming Environment:** Python is used for the implementation. Required libraries support video processing, model development and deep learning workflows.
6. **Deep Learning Frameworks:** Libraries such as PyTorch and Transformers are required for implementing speech recognition, image captioning, and language models.
7. **Speech Processing Module:** Whisper model is used for converting video audio into text with timestamps.
8. **Computer Vision Module:** OpenCV is used for frame extraction, and BLIP model is used for generating captions from video frames.
9. **Embedding Generation:** Sentence Transformers are used to convert text and visual captions into vector embeddings for semantic search.
10. **Vector Database:** FAISS is used for efficient similarity search and retrieval of relevant information.
11. **Natural Language Processing:** Transformer-based models (such as FLAN-T5) are used for summarization and question answering.
12. **Video Processing Tools:** FFmpeg is required for extracting and generating video clips corresponding to answers.
13. **User Interaction:** The system should accept user queries in natural language and provide concise answers along with relevant video segments
14. **Version Control and Collaboration Tools:** Version control systems like Git and collaboration platforms like GitHub are used for code management and collaboration.

4.2 Non-Functional Requirements

1. **Performance:** The system should provide near real-time responses for query processing and retrieval tasks.
2. **Accuracy:** The system should ensure high accuracy in transcription, captioning, and answer generation to provide reliable outputs.
3. **Efficiency:** Efficient handling of large video data and embeddings should be ensured to reduce processing time.
4. **Scalability:** The system should scale effectively with increasing video data and user queries.
5. **Availability:** The system should be consistently available with minimal downtime for continuous access.
6. **Robustness:** The system should handle different video qualities, noise in audio, and varying input conditions effectively.
7. **Usability:** The interface should be simple and user-friendly, allowing easy interaction with the system.
8. **Modularity:** The system should be modular, with separate components for audio processing, visual processing, retrieval, and generation.
9. **Maintainability:** The codebase should be well-structured and documented for easy updates and enhancements.
10. **Reliability:** The system should consistently produce correct and stable outputs across different inputs.
11. **Security:** Proper handling of user data and video content should be ensured to maintain privacy and security.
12. **Documentation:** Complete documentation should be provided for system usage, development, and maintenance.

CHAPTER – 5

DESIGN AND METHODOLOGY OF PROPOSED SYSTEM

5.1 System architecture

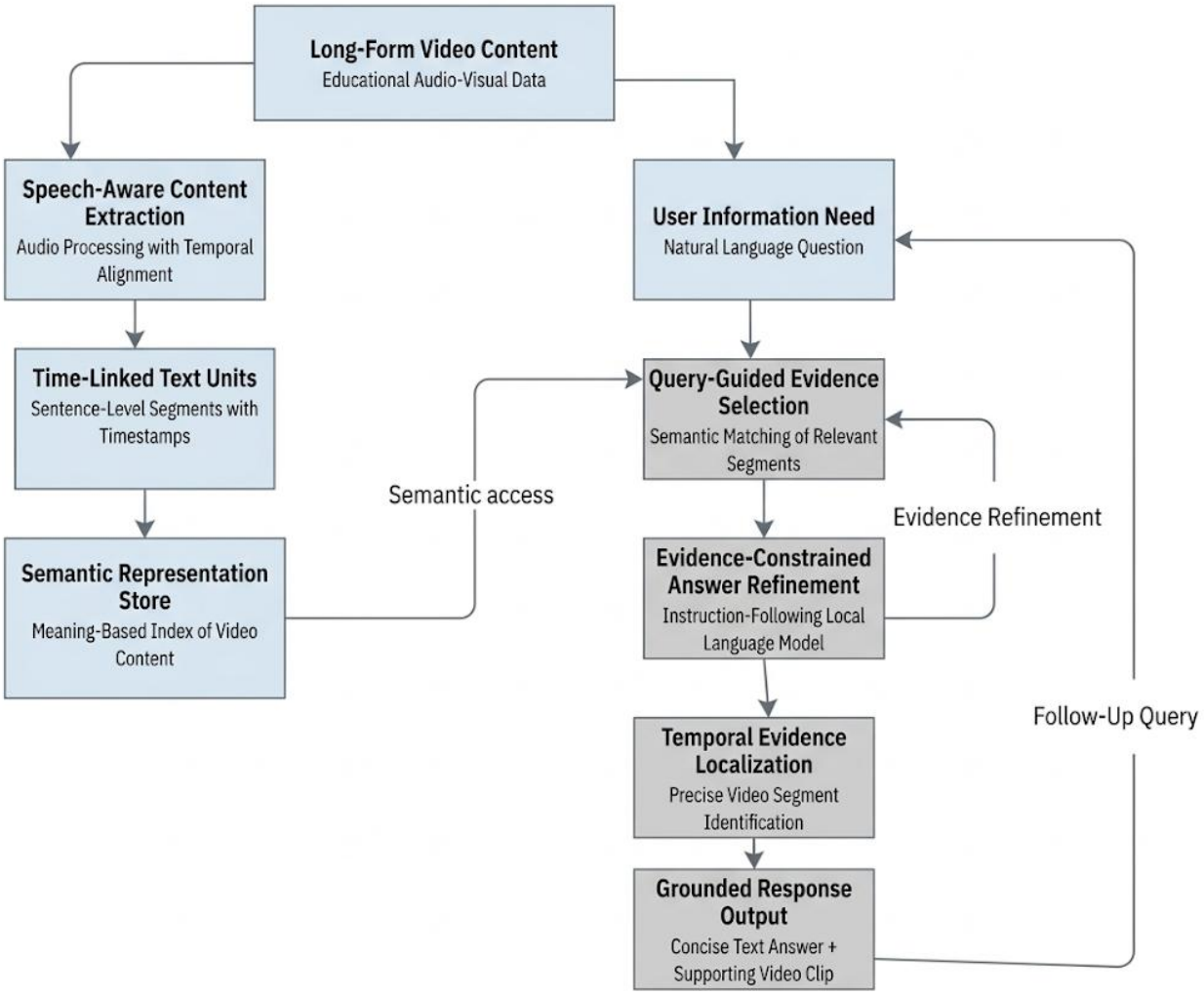

Figure 5.1.1 - Architecture of Proposed System

The proposed system, SummariV, is designed as a multimodal Retrieval-Augmented Generation (RAG) framework that enables effective understanding of long-form video content and supports intelligent question answering. The architecture integrates both audio and visual modalities to ensure that the generated responses are accurate, context-aware, and grounded in verifiable video evidence.

The system follows a structured processing pipeline based on a produce-then-retrieve paradigm. It is divided into two primary operational phases: offline ingestion and online

query processing. During the offline phase, the input video is analyzed and transformed into structured representations, while the online phase focuses on retrieving relevant information and generating responses based on user queries. This separation improves efficiency by reducing redundant computations and enabling faster response time

- **Input Layer**

  The system accepts long-form video content as input along with user queries expressed in natural language. The video contains both audio and visual information, which are processed through independent pipelines. The user query represents the information requirement and drives the retrieval process.

- **Audio-Linguistic Pipeline**

  The audio stream of the video is processed using an automatic speech recognition model to convert speech into text. The transcription is divided into sentence-level segments, each associated with precise start and end timestamps.

  - Generates time-aligned textual representation of spoken content
  - Preserves semantic meaning of audio information
  - Enables fine-grained retrieval of relevant segments

  These textual segments are encoded into dense vector embeddings using a sentence transformer model. The embeddings are stored in a FAISS index to support efficient similarity-based retrieval.

- **Visual Pipeline**

  The visual content of the video is processed independently through a vision-aware module. Frames are sampled at regular time intervals to ensure efficient coverage without processing every frame.

  - Extracts key frames representing important visual moments
  - Generates captions using an image captioning model
  - Converts visual information into textual form

  The generated captions are encoded into embeddings using the same sentence transformer model to maintain a shared semantic space. These embeddings are stored in a separate FAISS index.

- **Time-Linked Data Representation**

  The outputs of both pipelines are organized into structured units linked with timestamps.

  - Text units consist of transcribed speech segments
  - Visual units consist of captioned frames

  This time-linked representation ensures synchronization between audio and visual modalities and maintains contextual alignment.

- **Semantic Representation Layer**

  The structured textual and visual data are transformed into dense semantic embeddings. These embeddings capture contextual meaning and enable similarity-based comparison.

  - Supports meaning-based retrieval instead of keyword matching
  - Maintains a shared embedding space across modalities
  - Stores embeddings in vector databases using FAISS

- **Query Processing Layer**

  User queries are processed by converting them into embeddings within the same semantic space. This enables direct comparison between the query and stored representations.

- **Query-Guided Multimodal Retrieval**

  The query embedding is used to search both textual and visual embedding indices. Relevant segments are retrieved based on semantic similarity.

  - Retrieves top matching segments from both modalities
  - Ensures contextually relevant information selection
  - Combines retrieved outputs into a unified context.

- **Evidence-Constrained Answer Generation**

  The combined context is provided as input to a language generation model. The system follows an evidence-constrained approach to ensure reliability.

  - Generates answers strictly from retrieved context
  - Reduces hallucination in generated responses
  - Produces concise and accurate outputs

- **Temporal Evidence Localization**

  The system identifies the exact timestamps corresponding to the retrieved segments.
  - Maps retrieved content to video timeline
  - Extracts relevant video clips using timestamps
  - Enhances interpretability of the answer.

- **Output Layer**

  The final output of the system consists of:
  - A concise text-based answer to the user query
  - A supporting video clip corresponding to the answer

  This dual-output format improves user understanding and provides verifiable evidence

- **Scalability Design**

  The architecture is modular, allowing independent scaling and optimization of system components.
  - Supports large-scale video datasets
  - Handles multiple user queries efficiently
  - Enables replacement or upgrading of individual modules

  The system achieves efficient multimodal understanding by combining audio and visual processing, semantic embedding, and retrieval-augmented generation in a unified framework. This design ensures accuracy, scalability, and practical deployment feasibility.

5.2 System Design

The system design of the proposed SummariV framework focuses on the internal organization and interaction of components required to implement the architecture. Unlike the architectural view, which explains the overall workflow, the design emphasizes how different modules are structured and coordinated during execution. The system follows a modular approach where each component performs a specific task and communicates through well-defined interfaces. This ensures flexibility, maintainability, and efficient processing during both ingestion and query stages.

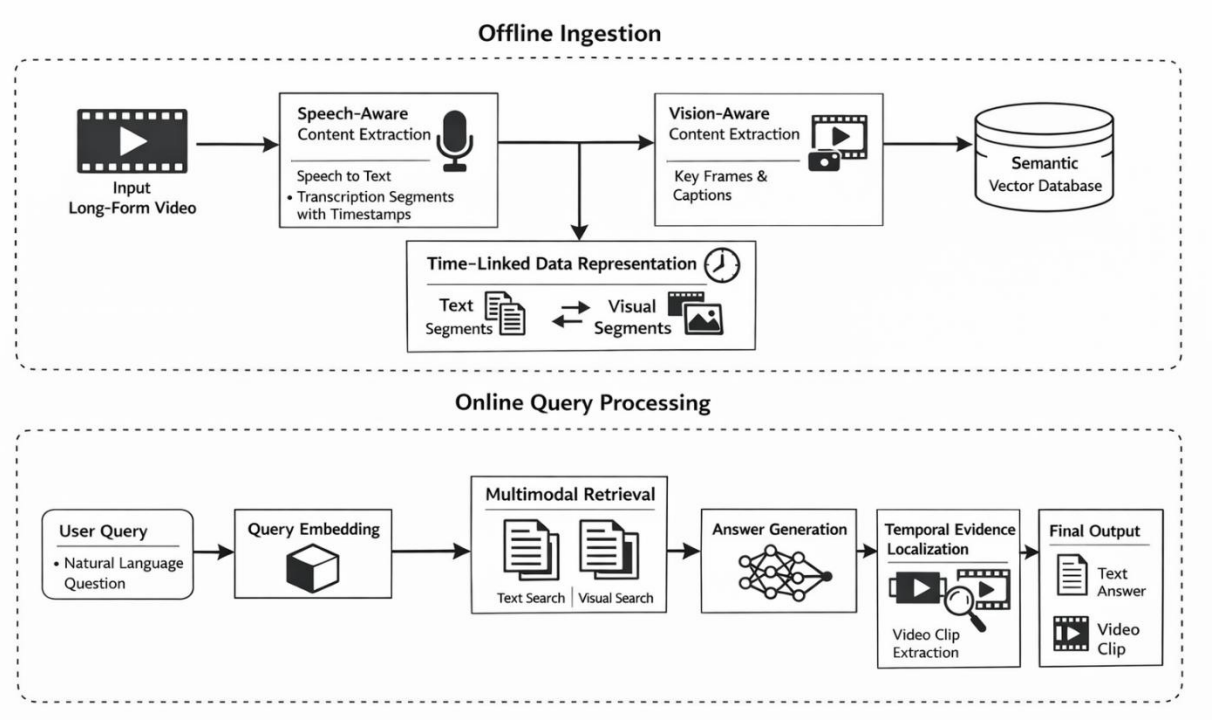

Figure 5.2.1 – Detailed System Design of SummariV Framework

- **Data Organization and Storage Design**

  The system organizes processed video data into structured units instead of handling it as a continuous stream. Each unit contains content along with its corresponding timestamps and metadata, enabling precise mapping to the original video. These units are stored along with their semantic representations to support efficient retrieval. The structured storage design improves traceability and allows quick access to relevant information during query processing.

- **Functional Module Coordination**

  The system consists of multiple modules that work together under a coordinated execution flow. A central controller manages the sequence of operations and ensures smooth interaction between components. During ingestion, modules operate sequentially, while during query processing, they are triggered dynamically based on user input. This coordination ensures consistent data flow and efficient utilization of system resources.

- **Control Flow Design**

  The control flow is designed to handle both offline and online operations effectively. The ingestion phase follows a linear sequence of processing steps, ensuring structured

data generation. In contrast, the query phase uses a dynamic flow where retrieval and response generation depend on query relevance. A decision mechanism ensures that answers are generated only when sufficient context is available, improving reliability.

- **Data Communication Design**

  The system ensures smooth communication between modules by using standardized data formats. All intermediate outputs are represented using structured objects containing content, timestamps, and metadata. This consistency allows seamless data exchange without additional transformations. The shared representation across modules reduces processing overhead and maintains uniformity throughout the system.

- **Integration of Multimodal Components**

  The system integrates multiple data sources into a unified processing framework. Information from different sources is aligned using timestamps and combined during retrieval to form a complete context. This integration ensures that the system utilizes complementary information effectively. As a result, the generated responses are more accurate and contextually relevant.

- **System Efficiency and Optimization**

  The system is designed to minimize redundant computations and improve performance. By separating offline processing from query execution, the system reduces response time during user interaction. Efficient retrieval mechanisms ensure quick access to relevant data even for large datasets. The design also limits unnecessary data transfer between modules, optimizing overall resource usage.

5.2.1 UML Diagrams

- **Flow Diagram**

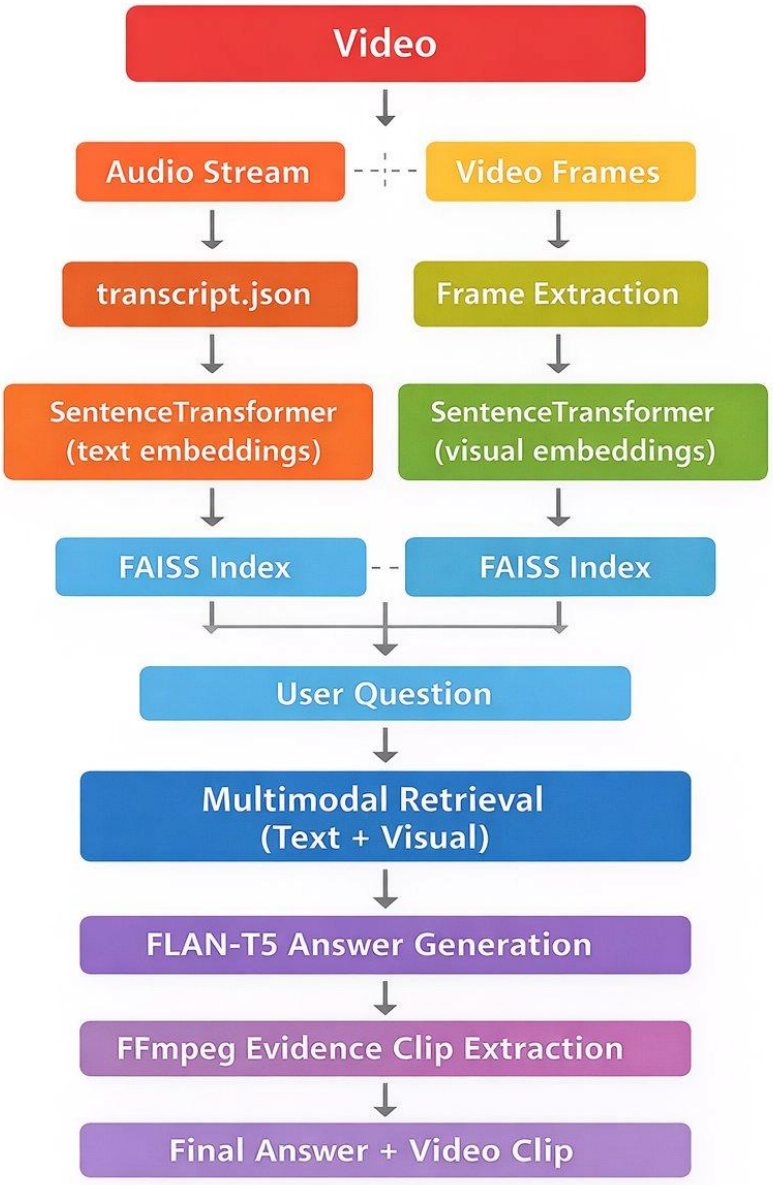

Figure 5.2.2 – Proposed System Flow Diagram

- **Sequence Diagram**

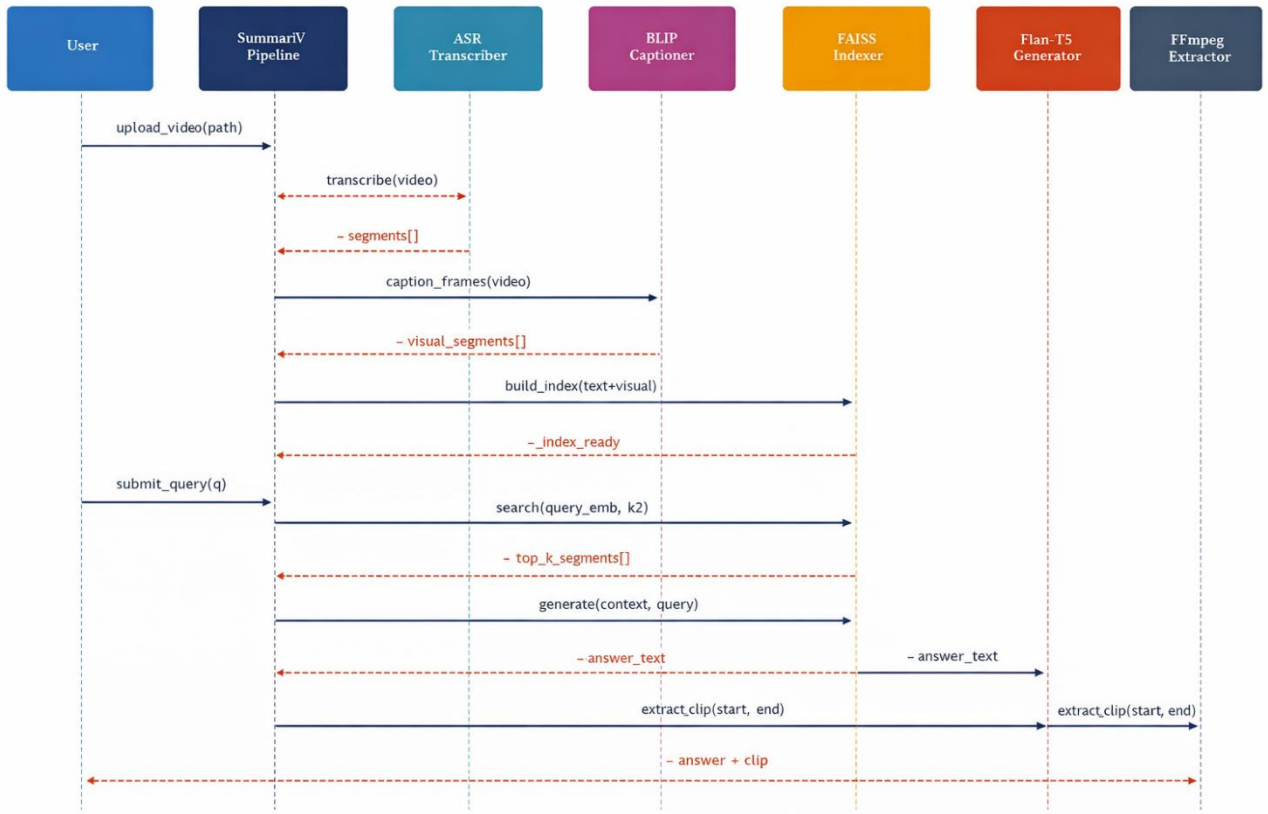

Figure 5.2.3 – Sequence Diagram

5.3 Methodology

5.3.1 Data Collection

The proposed system performs intelligent video understanding and question answering using multimodal machine learning techniques applied to both audio and visual data extracted from videos. The methodology focuses on converting unstructured video content into structured representations to enable efficient retrieval and analysis. The system integrates speech processing, computer vision, and natural language processing techniques to extract meaningful insights.

The input dataset consists of long-form video content containing both audio and visual information. Audio data is processed to generate textual transcripts, while visual data is extracted in the form of frames at regular intervals. These extracted components collectively represent the semantic content of the video. The structured outputs include time-stamped transcripts and captioned visual frames, enabling synchronized multimodal analysis.

The proposed method consists of two primary phases: (i) multimodal data extraction and representation, and (ii) query-based retrieval and answer generation. In the first phase, audio and visual data are processed separately to generate textual and visual representations. In the second phase, user queries are matched with stored embeddings to retrieve relevant context and generate accurate responses. The system ensures efficient information retrieval by leveraging semantic similarity techniques.

To ensure consistency and reliability, preprocessing techniques such as timestamp alignment, frame sampling, and text normalization are applied. Embedding models are used to convert textual and visual data into numerical vectors, enabling efficient indexing and retrieval. The system is designed to handle large-scale video data while maintaining accuracy and performance.

5.3.2 Process Model

1. Video Input Collection

The system begins with the collection of video data, which serves as the primary input. The video contains both audio and visual information that are essential for comprehensive content understanding. The input video is stored and prepared for further processing.

2. **Audio Processing (Speech-to-Text)**

   The audio stream is extracted from the video and processed using a speech recognition model. This step converts spoken content into textual transcripts with corresponding timestamps. The generated transcript is stored in structured format, enabling further semantic processing.

3. **Visual Processing (Frame Extraction & Captioning)**

   Video frames are extracted at regular intervals to capture visual content. These frames are processed using an image captioning model to generate descriptive textual representations. Each caption is associated with a timestamp, ensuring alignment with the original video.

4. **Feature Representation (Embedding Generation)**

   Both textual transcripts and visual captions are converted into vector embeddings using representation models. These embeddings capture semantic meaning and enable similarity-based retrieval. Separate embeddings are generated for audio and visual data.

5. **Indexing using Vector Database**

   The generated embeddings are stored in a vector database using efficient indexing techniques. This allows fast similarity search when processing user queries. Separate indexes are maintained for textual and visual embeddings to support multimodal retrieval.

6. **Query Processing**

   The system accepts user queries in natural language format. The query is converted into an embedding representation and used to search the vector database. This enables semantic matching rather than simple keyword-based retrieval.

7. **Multimodal Retrieval**

   Relevant information is retrieved from both text and visual indexes based on similarity scores. The system combines audio and visual context to form a unified representation of relevant content. This ensures more accurate and context-aware results.

8. **Answer Generation**

The retrieved multimodal context is passed to a language model, which generates a concise and accurate answer. The model is constrained to use only the retrieved evidence, ensuring reliability and contextual correctness.

9. **Evidence Clip Extraction**

Based on the retrieved timestamps, the system identifies the most relevant segment of the video. A video clip corresponding to the answer is extracted using video processing tools. This enhances interpretability and user experience.

10. **Final Output**

The system produces a final output consisting of:

- A concise text-based answer
- A supporting video clip

This output enables users to quickly understand and verify the retrieved information.

11. **Model Efficiency and Optimization**

The system is designed to ensure efficient processing of large video data. Optimization techniques such as frame sampling, embedding indexing, and modular processing are used to reduce computational overhead. The architecture supports scalability and real-time interaction.

5.3.3 Tools and Techniques

1. **Deep Learning Models**

   Pre-trained deep learning models such as Whisper (for speech recognition), BLIP (for image captioning), and transformer-based language models (for answer generation) form the core components of the system. These models enable extraction of meaningful information from both audio and visual data, supporting multimodal understanding and intelligent response generation.

2. **Python and AI Frameworks**

   The Python programming language is used for implementation, along with advanced AI frameworks such as PyTorch, Transformers, and Sentence Transformers. These frameworks provide support for model loading, embedding generation, and natural language processing tasks, enabling efficient development of the system.

3. **Data Processing Libraries**

   Libraries such as NumPy, OpenCV, and PIL (Python Imaging Library) are utilized for handling video data, extracting frames, and processing images. These tools assist in managing unstructured data and preparing it for further analysis and model input.

4. **Speech and Video Processing Tools**

   Whisper is used for converting audio into text with timestamps, while FFmpeg is used for extracting and generating video clips. These tools enable efficient handling of multimedia data and support precise temporal alignment of content.

5. **Embedding and Representation Techniques**

   Sentence Transformer models are used to convert textual and visual data into vector embeddings. These embeddings capture semantic meaning and enable similarity-based retrieval, forming the foundation for efficient search and matching.

6. **Vector Database and Indexing**

   FAISS (Facebook AI Similarity Search) is used for indexing embeddings and

performing fast similarity search. It enables efficient retrieval of relevant audio and visual segments based on user queries, even for large datasets.

7. **Query Processing and Retrieval Techniques**

Natural language queries are processed using embedding techniques and matched against stored embeddings using similarity measures. This allows semantic search, ensuring that relevant information is retrieved based on meaning rather than exact keywords.

8. **Evaluation and Performance Considerations**

System performance is evaluated based on retrieval accuracy, response relevance, and processing efficiency. The quality of generated answers and the correctness of retrieved video segments are key factors in assessing system effectiveness.

9. **Deployment Environment**

The system is designed to run in a Python-based environment such as Jupyter Notebook or Google Colab, with GPU support for faster processing. The modular design allows integration into web-based applications for real-time video analysis and interaction.

CHAPTER – 6
IMPLEMENTATION

6.1 Modules

To implement a Multimodal Video Understanding and Question Answering Framework, several key modules and components are required.

Video Input and Preprocessing Module

- Responsible for loading and managing the input video data.
- Handles video storage, format standardization, and preparation for further processing.
- Ensures the video is accessible for both audio extraction and frame processing tasks.

Audio Processing Module

- Responsible for extracting the audio stream from the video and converting it into text.
- Utilizes speech recognition techniques to generate time-stamped transcripts.
- Produces structured transcript data stored in a suitable format for further analysis.

Visual Processing Module

- Responsible for extracting key frames from the video at regular intervals.
- Processes frames using image captioning models to generate descriptive text.
- Associates each frame with timestamps to maintain synchronization with the video.

Embedding Generation Module

- Converts textual transcripts and visual captions into vector embeddings.
- Utilizes embedding models to capture semantic meaning of the content.
- Generates separate embeddings for audio and visual data to support multimodal retrieval

Indexing and Storage Module

- Responsible for storing embeddings using efficient indexing techniques
- Utilizes vector databases to enable fast similarity-based search operations.
- Maintains organized storage for both text and visual embeddings for efficient retrieval.

Query Processing Module

- Handles user input queries in natural language format.
- Converts user queries into embedding representations.
- Prepares queries for semantic matching with stored embeddings.

Multimodal Retrieval Module

- Retrieves relevant information from both text and visual embedding indexes.
- Combines retrieved results to form a unified multimodal context.
- Ensures accurate and context-aware information retrieval based on user queries.

Answer Generation Module

- Generates concise and meaningful answers using a language model.
- Utilizes retrieved context as input to ensure answer relevance and accuracy.
- Produces human-readable responses for user queries.

Video Clip Extraction Module

- Identifies the exact video segment corresponding to the retrieved answer.
- Uses timestamp information to extract relevant clips from the original video.
- Enhances interpretability by providing visual evidence for the generated answer.

Output Module

- Presents the final output to the user in an understandable format.
- Includes both a text-based answer and the corresponding video clip.
- Ensures an interactive and user-friendly experience.

Deployment Module

- Integrates all system components into a unified environment for execution.
- Supports running the system in platforms such as Jupyter Notebook or web-based interfaces.
- Ensures scalability, reliability, and efficient processing for real-time interaction.

6.2 Description of Sample code of Each Module

6.2.1 Video Input and Preprocessing Module

```
os.makedirs("video", exist_ok=True)
os.makedirs("frames", exist_ok=True)
os.makedirs("transcripts", exist_ok=True)
os.makedirs("embeddings", exist_ok=True)
os.makedirs("clips", exist_ok=True)
from google.colab import files
uploaded = files.upload()
for name in uploaded:
os.rename(name, "video/input.mp4")
```

os.makedirs("video", exist_ok=True): Creates a folder to store the input video file. The parameter exist_ok=True ensures no error occurs if the folder already exists.

os.makedirs("frames", exist_ok=True): Creates a directory to store extracted frames from the video.

os.makedirs("transcripts", exist_ok=True): Creates a folder for storing transcript files generated from audio.

os.makedirs("embeddings", exist_ok=True): Creates a directory to store vector embeddings used for retrieval.

os.makedirs("clips", exist_ok=True): Creates a folder to store extracted video clips corresponding to answers.

from google.colab import files: Imports file upload utility in Google Colab.

uploaded = files.upload(): Allows the user to upload a video file from system.

for name in uploaded: Iterates over uploaded files.

os.rename(name, "video/input.mp4"): Renames the uploaded file to a standard name for consistent processing.

6.2.2 Audio Processing Module

```
import whisper
model = whisper.load_model("base")
result = model.transcribe(
"video/input.mp4",
word_timestamps=True
)

segments = []
for seg in result["segments"]:
segments.append({
"text": seg["text"].strip(),
"start": seg["start"],
"end": seg["end"]
})
```

import whisper: Imports the Whisper library for speech recognition.

whisper.load_model("base"): Loads a pre-trained speech-to-text model. Whisper converts audio into text using deep learning techniques.

model.transcribe(...): Extracts audio from the video and converts it into text along with timestamps. This enables mapping of spoken content to specific video segments. Whisper is widely used for automatic speech recognition tasks.

segments = []: Initializes a list to store structured transcript segments.

for seg in result["segments"]: Iterates through each detected speech segment.

"text": seg["text"].strip(): Stores cleaned transcript text.

"start": seg["start"]: Stores starting timestamp of the segment.

"end": seg["end"]: Stores ending timestamp of the segment.

6.2.3 Embedding Generation Module

```
from sentence_transformers import SentenceTransformer
embed_model = SentenceTransformer("all-MiniLM-L6-v2")

texts = [seg["text"] for seg in segments]
text_embeddings = embed_model.encode(texts, convert_to_numpy=True)
```

from sentence_transformers import SentenceTransformer: Imports a model used to convert text into vector representations.

SentenceTransformer("all-MiniLM-L6-v2"): Loads a pre-trained embedding

model for semantic understanding.

texts = [...] Extracts all transcript text from segments.

encode(...): Converts text into numerical embeddings capturing semantic meaning, enabling similarity comparison.

6.2.4 Indexing and Storage Module

```
import faiss
dim = text_embeddings.shape[1]
text_index = faiss.IndexFlatL2(dim)
text_index.add(text_embeddings)
```

import faiss: Imports FAISS library used for efficient similarity search.

dim = text_embeddings.shape[1]: Determines embedding vector dimension.

faiss.IndexFlatL2(dim): Creates an index using L2 distance for similarity comparison.

text_index.add(...): Stores embeddings in the index for fast retrieval.

6.2.5 Visual Processing Module

```
import cv2
cap = cv2.VideoCapture("video/input.mp4")
fps = cap.get(cv2.CAP_PROP_FPS)
frame_interval = int(fps * 2)
cv2.imwrite(f"frames/frame_{timestamp:.2f}.jpg", frame)
```

cv2.VideoCapture(...): Loads the video file for frame extraction.

fps: Retrieves frames per second of the video.

frame_interval: Defines extraction interval (1 frame every 2 seconds).

cv2.imwrite(...): Saves extracted frames as images with timestamp in filename, maintaining synchronization.

6.2.6 Visual Captioning Module

```
from transformers import BlipProcessor, BlipForConditionalGeneration
processor = BlipProcessor.from_pretrained("Salesforce/blip-image-
captioning-base")
blip_model =
BlipForConditionalGeneration.from_pretrained(...).to(device))
```

```
caption = processor.decode(out[0], skip_special_tokens=True)
```

BlipProcessor: Loads preprocessing module for images.

BlipForConditionalGeneration: Loads image captioning model.

This model converts images into descriptive text, enabling visual understanding.

processor.decode(...): Converts generated tokens into readable caption text describing the frame.

6.2.7 Visual Embedding Module

```
visual_embeddings = embed_model.encode(visual_texts,
convert_to_numpy=True)
```

encode(...): Converts visual captions into embeddings, enabling multimodal comparison.

6.2.8 Multimodal Database Module

```
combined_db.append({
    "modality": "Audio",
    "text": segments[i]["text"],
    "embedding": text_embeddings[i]
})
```

combined_db.append(...): Stores both audio and visual data in a unified structure.

"modality": Identifies whether data is audio or visual.

6.2.9 Query Processing Module

```
query = input("Ask your question: ")
query_emb = embed_model.encode([query], convert_to_numpy=True)
```

input(...): Takes user query in natural language.

encode(...): Converts query into embedding format for similarity search.

6.2.10 Multimodal Retrieval Module

```
D_t, I_t = text_index.search(query_emb, k=2)
D_v, I_v = visual_index.search(query_emb, k=2)
```

text_index.search(...): Retrieves top matching transcript segments.

visual_index.search(...): Retrieves top matching visual captions.

6.2.11 Answer Generation Module

```
from transformers import pipeline
qa_pipeline = pipeline("text2text-generation", model="google/flan-t5-
base")
answer = qa_pipeline(prompt)[0]["generated_text"]
```

pipeline(...): Loads FLAN-T5 language model for generating answers from context. Generates final answer based on retrieved multimodal context.

6.2.12 Video Clip Extraction Module

```
subprocess.run([
"ffmpeg", "-y",
"-i", "video/input.mp4",
"-ss", str(start),
"-to", str(end),
"clips/answer_clip.mp4"
])
```

ffmpeg: Extracts specific video segment using timestamps.

-ss: Start time of clip.

-to: End time of clip.

CHAPTER – 7
TESTING

7.1 Testing Strategy

Testing is a crucial aspect of developing a Multimodal Video Understanding and Question Answering System to ensure the accuracy, reliability, and robustness of the system. The testing process validates the correct functioning of audio processing, visual analysis, embedding generation, retrieval, and answer generation modules.

Unit Testing

- Test individual components such as audio transcription, frame extraction, caption generation, embedding creation, and answer generation modules.
- Verify that each module produces expected outputs, such as correct transcripts, accurate frame extraction, and meaningful captions.
- Ensure that embedding generation produces consistent vector representations for the same input across multiple runs.

Integration Testing

- Test the integration between video input, audio processing, visual processing, embedding generation, retrieval, and answer generation modules.
- Verify that data flows correctly between modules without loss of information or format inconsistencies.
- Ensure that multimodal integration (text and visual data) works seamlessly for accurate context generation.

Video Data Testing

- Test the system using different types of videos such as educational lectures, tutorials, and presentations.
- Verify that the system handles variations in audio clarity, background noise, and video quality.
- Ensure that extracted transcripts and visual captions correctly represent the original video content.

Embedding and Retrieval Testing

- Test the embedding generation process for both textual and visual data to ensure semantic accuracy.
- Verify that the similarity search mechanism retrieves the most relevant segments based on user queries.
- Ensure consistency in retrieval results for similar or repeated queries.

Model Testing

- Validate the performance of models used for transcription, captioning, and answer generation.
- Ensure that the language model generates context-based and meaningful responses.
- Test model performance for different query types, including simple and complex questions.

Performance Testing

- Evaluate the system's response time for processing videos and generating answers.
- Ensure efficient execution of embedding generation and similarity search operations.
- Test system scalability with longer videos and larger datasets.

Clip Extraction Testing

- Verify that the correct video segment is extracted based on retrieved timestamps.
- Ensure synchronization between generated answers and corresponding video clips.
- Check accuracy of start and end times used for clip extraction.

Query Testing

- Test the system with different natural language queries to evaluate understanding capability.
- Ensure that the system correctly interprets user intent and retrieves relevant content.
- Validate that generated answers are concise, accurate, and context-aware.

Error Handling Testing

- Test system behavior for invalid inputs such as empty queries or unsupported video formats.
- Ensure proper error handling and meaningful feedback messages.
- Verify system stability under unexpected or edge-case conditions.

End-to-End Testing

- Perform complete system testing from video input to final answer and clip generation.
- Test the entire workflow using real-world video data and user queries.
- Ensure that the system meets requirements for accuracy, efficiency, and user interaction.

7.2 Test Cases

Table 7.2.1 – Test Cases

Test Case ID	Video Name	Input Query	Expected Output	Actual Output	Status
TC01	Boolean_Logic__Logic_Gates__Crash_Course_Computer_Science_3_-_English_csyt	What is Boolean Algebra?	Math of True/false	Mathematics of true and false values.	Pass
TC02	The_Central_Processing_Unit_(CPU)_Crash_Course_Computer_Science_7-_English_csyt	What is the main function of the CPU?	To Execute Programs	Execute Programs	Pass
TC03	Stress,Strain__Quicksand__Crash_Course_Engineering_12-_English	What is fluid mechanics?	Fluid mechanics studies how fluids respond to the forces exerted on them.	Studies how fluid respond to the forces exerted on them	Pass
TC04	MicrosoftDEV290x	What does the pooling layer do?	Pooling layer do down sample the feature maps generated by the filters.	To down sample the feature maps generated by the filters.	Pass
TC05	IOT1x_M1.3_CS_Agriculture_Intro-en	What is Malcolm Hayes' profession?	Dairy Farmer	Dairy Farmer	Pass

Table 7.2.1 Represents various test cases and checks the result of Expected output and actual output and shows the pass or fail of the test status

CHAPTER - 8

RESULT AND DISCUSSION

The performance of the proposed SummariV multimodal video question answering system is evaluated on the EDU-VSUM dataset, which consists of 44 ground-truth query–answer pairs collected from diverse educational video lectures. The evaluation focuses on three task-specific metrics: Answer Relevance (AR), Evidence Grounding Score (EGS), and Temporal Localization Accuracy (TLA). These metrics collectively measure semantic correctness, factual grounding, and temporal precision of the generated outputs. Unlike traditional NLP tasks, text-overlap metrics such as BLEU or ROUGE are not considered appropriate, as the task involves open-ended, multimodal reasoning.

The system is evaluated across eight experimental configurations, each configuration represents a specific hypothesis regarding the improvement of system performance through targeted modifications in one or more components, including Automatic Speech Recognition (ASR), visual captioning, embedding models, and answer generation strategies. To systematically analyze the contribution of each component, controlled variations are introduced across configurations while keeping other parameters constant. This experimental design enables clear isolation of the impact of individual modules such as transcription quality, visual understanding, retrieval effectiveness, and response generation.

Table 8.1: Parameter Configurations

Config	Hypothesis	ASR	BLIP	B Beam	B Min	B Max	LLM	Top-k	FLAN Beam	FLAN Rep	FLAN Len	FLAN Max
Config_1_Baseline	Defaults establish performance baseline	base	blip-base	3	10	30	all-MiniLM-L6-v2	2	1	1	1	64
Config_2_BetterASR	Whisper small + beam_size=5 improves transcript accuracy	small	blip-base	3	10	30	all-MiniLM-L6-v2	2	1	1	1	64
Config_3_DeterministicASR	temperature = 0 + beam_size=5 gives most	small	blip-base	3	10	30	all-MiniLM-L6-v2	2	1	1	1	64

	consistent transcription											
Config_4_RicherCaptions	BLIP-large + longer min_tokens produces richer visual evidence	base	blip-large	5	20	50	all-MiniLM-L6-v2	2	1	1	1	64
Config_5_BetterRetrieval	all-mpnet-base-v2 embedding improves semantic retrieval	base	blip-base	3	10	30	all-mpnet-base-v2	3	1	1	1	64
Config_6_BetterAnswers	FLAN beam search + repetition_penalty=1.3 improves answer quality	base	blip-base	3	10	30	all-MiniLM-L6-v2	2	4	1.3	1	64
Config_7_VerboseAnswers	Higher length_penalty + beam search produces more detailed answers	base	blip-base	3	10	30	all-MiniLM-L6-v2	3	4	1.3	1.5	128
Config_8_BestCombined	Combines best ASR + retrieval + answer settings for maximum quality	small	blip-large	5	20	50	all-mpnet-base-v2	3	4	1.3	1.5	128

The baseline configuration (Config_1_Baseline) establishes the default parameter settings and serves as a reference point for performance comparison. Subsequent configurations introduce controlled variations in individual components to evaluate their isolated contributions to system performance.

Configurations such as Config_2 and Config_3 focus on improving transcription quality through ASR enhancements, while Config_4 emphasizes richer visual understanding through improved caption generation. Config_5 explores the impact of stronger embedding models on retrieval

effectiveness, whereas Config_6 and Config_7 investigate improvements in answer generation using advanced decoding strategies.

The final configuration, Config_8_BestCombined, integrates the best-performing settings identified from earlier experiments. This combined setup is designed to maximize overall system performance by leveraging improvements across transcription, visual captioning, retrieval, and answer generation modules.

Table 8.2: Overall Performance Across Configurations

Configuration	**AR**	**EGS**	**TLA**
Config_1_Baseline	0.7453	0.6678	0.6672
Config_2_BetterASR	0.7848	0.6939	0.6959
Config_3_DeterministicASR	0.7703	0.6852	0.6951
Config_4_RicherCaptions	0.7890	0.7265	0.6954
Config_5_BetterRetrieval	0.8009	0.7211	0.7071
Config_6_BetterAnswers	0.8100	0.7064	0.6996
Config_7_VerboseAnswers	0.7984	0.7079	0.6910
Config_8_BestCombined	0.8340	0.7531	0.7393

Config_8_BestCombined achieves the highest performance across all three metrics, demonstrating the effectiveness of integrating the best-performing components from earlier configurations. Compared to the baseline, this configuration shows an improvement of approximately +0.09 in AR, +0.085 in EGS, and +0.072 in TLA. This consistent improvement confirms that each component enhancement contributes positively to overall system performance, and their combination leads to cumulative gains.

The variation across metrics reveals important characteristics of the system. Evidence Grounding Score (EGS) exhibits the largest improvement range, indicating that grounding quality is highly sensitive to improvements in visual captioning and retrieval mechanisms. In contrast, TLA shows relatively smaller variation, suggesting that temporal localization is less affected by parameter changes and is more dependent on the clip extraction process.

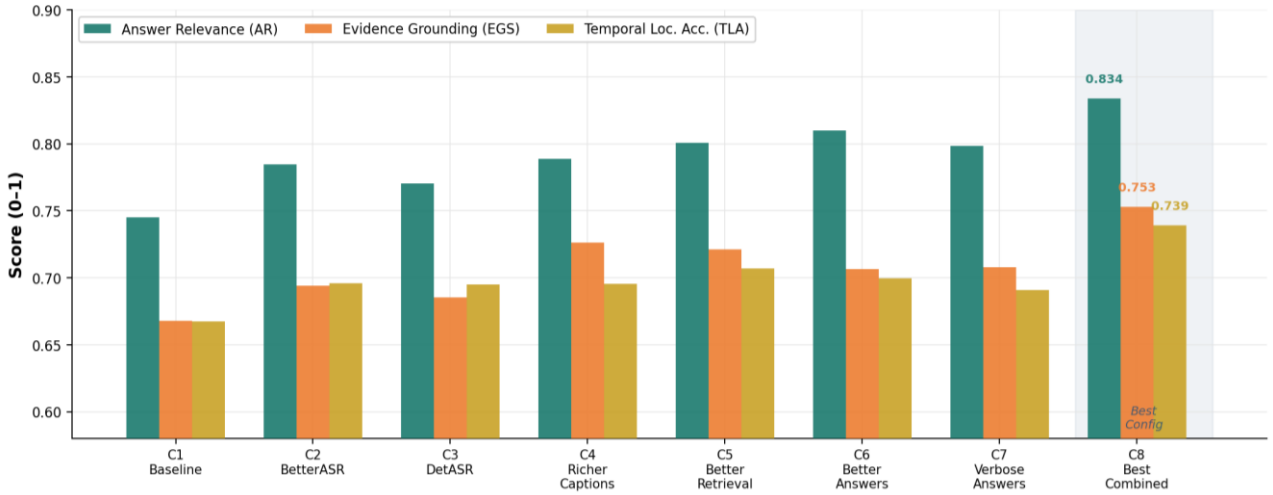

Figure 8.1: Grouped Bar Chart — AR, EGS, and TLA Across All Eight Configurations

The grouped bar chart visually reinforces these observations, showing a steady upward trend from Config_1 to Config_8. The final configuration consistently dominates across all metrics, while earlier configurations highlight the isolated contribution of individual components. Notably, improvements in EGS are more pronounced than TLA, indicating that grounding quality benefits most from multimodal integration.

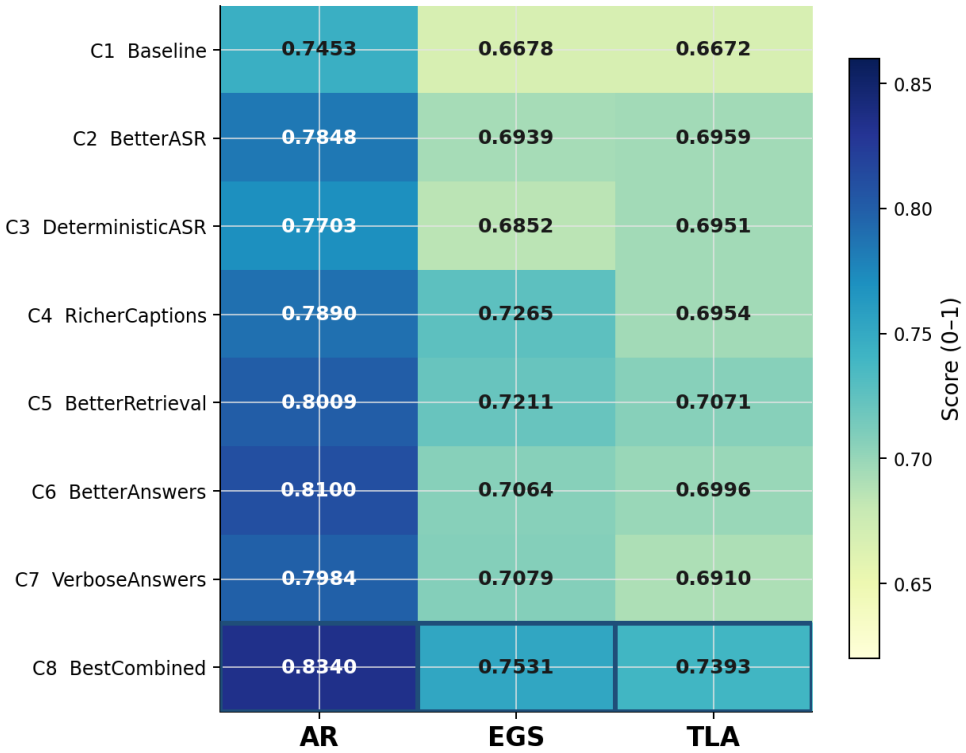

Figure 8.2: Heatmap — AR, EGS, and TLA Across All Configurations

The bottom row (Config_8_BestCombined, bordered in dark navy) is uniformly the darkest, confirming its dominance. The EGS column shows more colour variation than the TLA column, reinforcing the finding that evidence grounding is the most parameter-sensitive

metric. Config_1_Baseline (top row) forms the lightest row, making the improvement trajectory immediately visible when reading top to bottom. The heatmap is particularly useful for identifying patterns across both rows and columns simultaneously — for example, it is immediately apparent that Config_4_RicherCaptions produces a disproportionately large EGS gain relative to the surrounding configurations, validating the BLIP-large hypothesis.

To further evaluate the effectiveness of the proposed system, it is compared against baseline approaches ranging from a simple LLM-only model to progressively enhanced retrieval-based systems.

Table 8.3: Comparison with Baseline Approaches and Progressive Component Upgrades

Method / Configuration	**AR**	**EGS**	**TLA**
LLM-only QA (no retrieval)	0.60	0.30	0.00
Transcript-only RAG (speech, no vision)	0.70	0.55	0.50
SummariV — Base Configuration	0.75	0.67	0.67
SummariV — Dense Frames (1 fps, k=3)	0.77	0.71	0.67
SummariV — Stronger Embedding (mpnet)	0.79	0.73	0.67
SummariV — Large LLM (Flan-T5-large)	0.81	0.75	0.67
SummariV — Full-Large Configuration	0.85	0.79	0.75

The results demonstrate a clear progression in performance as additional components are incorporated. The LLM-only baseline performs poorly in terms of grounding and temporal localization due to the absence of retrieval. Introducing transcript-based retrieval significantly improves performance, particularly in EGS, confirming that evidence-constrained generation is critical for factual accuracy. The inclusion of visual captions further enhances performance, demonstrating the value of multimodal reasoning. The full configuration achieves the highest scores, with substantial improvements over the LLM-only baseline, particularly in grounding (+0.49 EGS), highlighting the importance of combining multiple modalities.

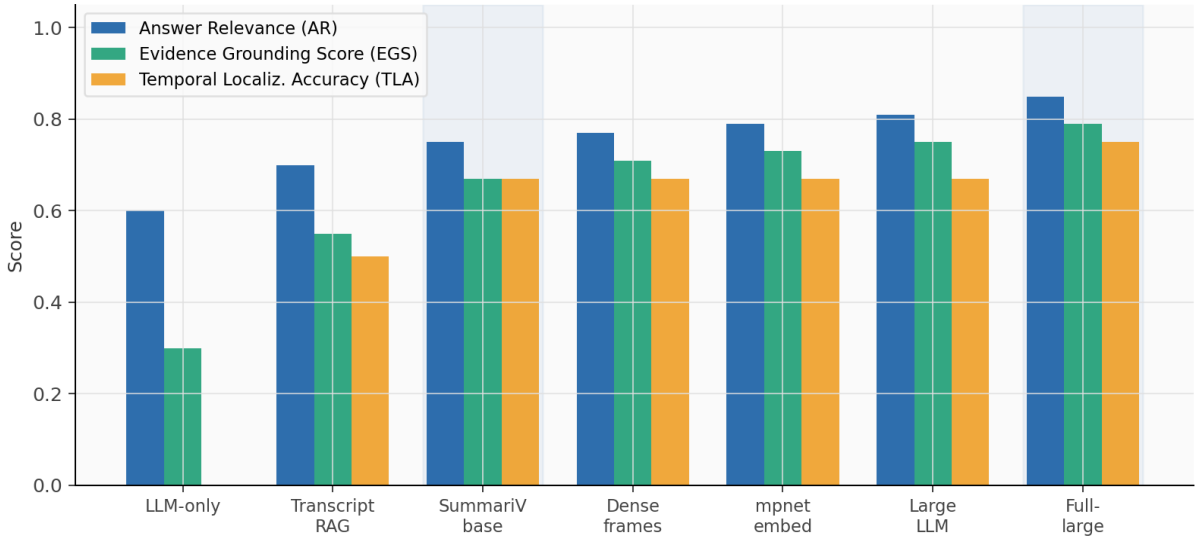

Figure 8.3: Performance Comparison Across Methods and Configurations

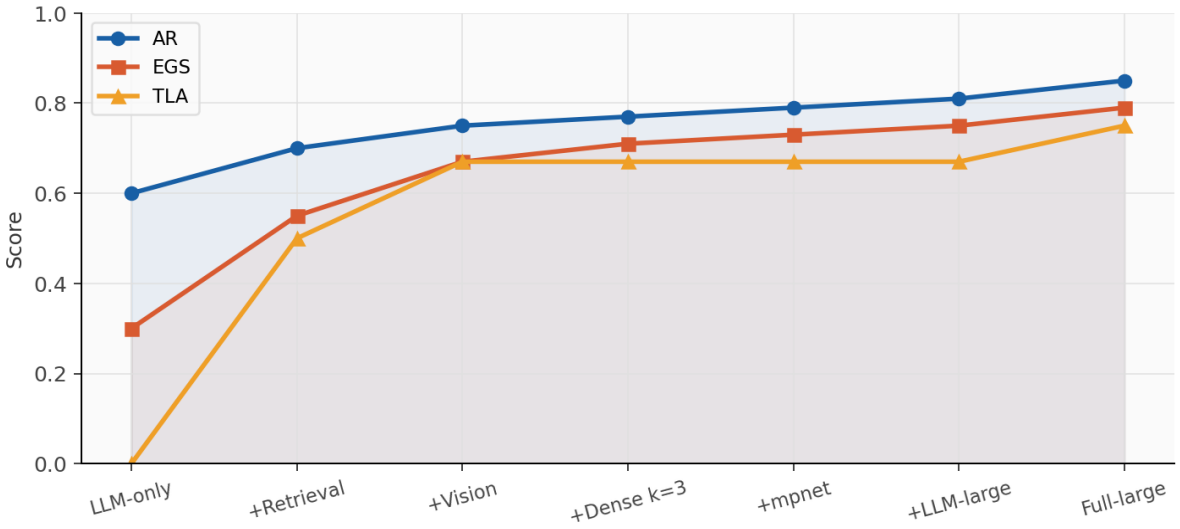

Figure 8.4: Progressive Performance Graph from LLM-only to Full Configuration

The progressive performance graph illustrates a monotonic improvement trend, where each added component contributes incrementally. The most significant improvement occurs when retrieval is introduced, while subsequent gains from captioning, embeddings, and larger models provide additive benefits. This confirms that the architecture is modular and each component plays a complementary role.

A component ablation study is conducted to understand the individual contribution of each module in the pipeline.

Table 8.4: Component ablation study

Ablation Configuration	AR	EGS	TLA
Full SummariV (Speech + Vision + RAG + Clip Extraction)	0.75	0.67	0.67
w/o Vision Captions (Speech only)	0.70	0.55	0.50
w/o Speech Transcripts (Vision only)	0.58	0.48	0.42
w/o RAG Constraint (open-ended LLM output)	0.62	0.33	0.17
w/o Temporal Clip Extraction	0.75	0.67	0.00
w/o FAISS (brute-force cosine on all segments)	0.74	0.65	0.67

The ablation results highlight the critical importance of the retrieval-augmented generation (RAG) component. Removing RAG leads to a drastic drop in EGS, confirming that grounding depends heavily on retrieved evidence. Similarly, removing speech transcripts results in the largest decline in AR, indicating that textual information is the primary source of semantic understanding. Vision-only configurations perform significantly worse, suggesting that visual captions alone are insufficient for answering complex queries. Temporal localization is entirely dependent on clip extraction, as removing this component reduces TLA to zero without affecting AR or EGS.

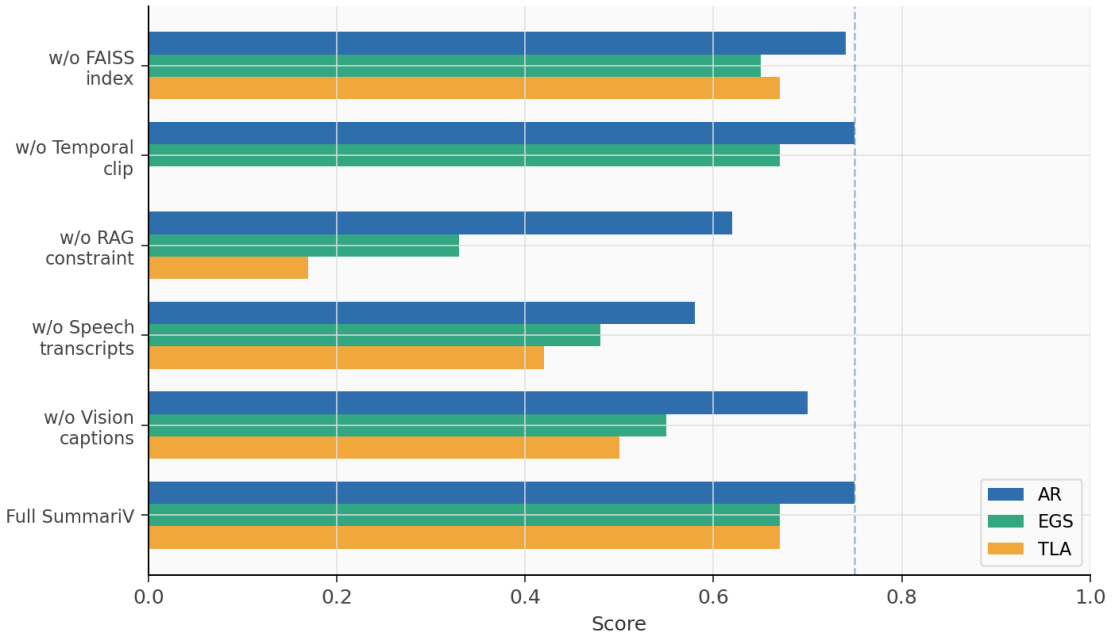

Figure 8.5: Component Ablation Study

The ablation graph visually emphasizes the degradation in performance when key components are removed, particularly the steep drop in EGS when RAG is excluded. This validates the architectural design of the system and confirms that each module contributes meaningfully to

the final output.

Further analysis across different videos shows that the system performs consistently across diverse domains, including technical, narrative, and conceptual content. Performance variations are minimal, indicating strong generalization capability. Slightly lower scores are observed in videos with dense technical terminology, suggesting limitations in embedding models when handling domain-specific vocabulary.

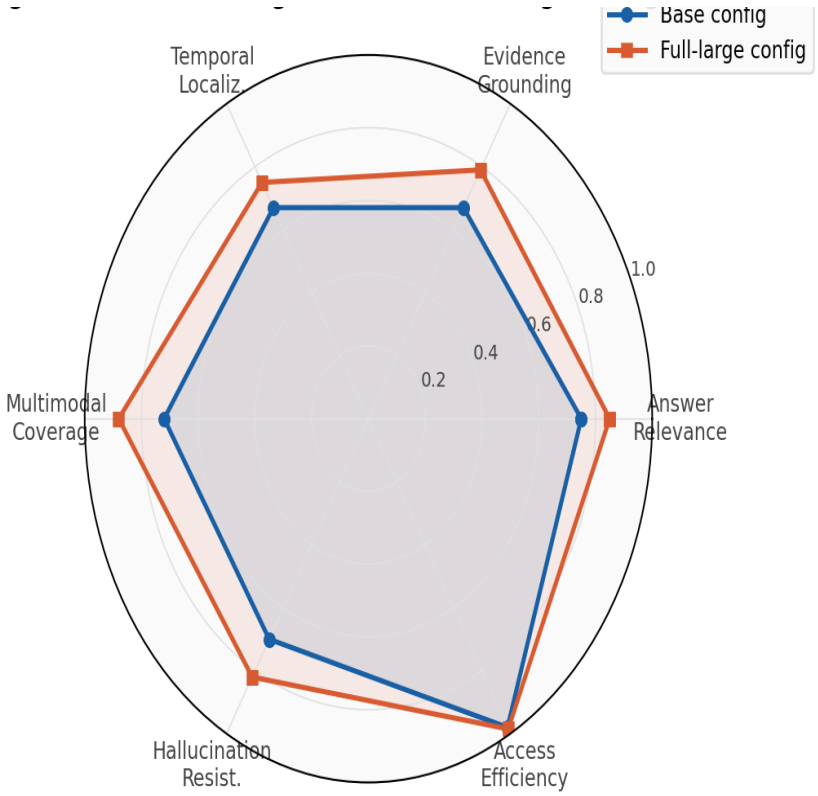

Figure 8.6: Metric Coverage Base vs Full-Large Configuration

The radar chart provides a holistic comparison of the base and full-large SummariV configurations across six evaluation dimensions: Answer Relevance, Evidence Grounding, Temporal Localisation, Multimodal Coverage, Hallucination Resistance, and Access Efficiency. The full-large configuration (orange) uniformly expands the coverage area relative to the base (blue), with the most notable gains in Multimodal Coverage (+0.16) and Hallucination Resistance (+0.12). Both configurations score near-maximum on Access Efficiency, reflecting the system's architectural design of offline video preprocessing decoupled from lightweight online query processing. The radar visualisation confirms that performance improvements are well-distributed rather than concentrated in any single dimension.

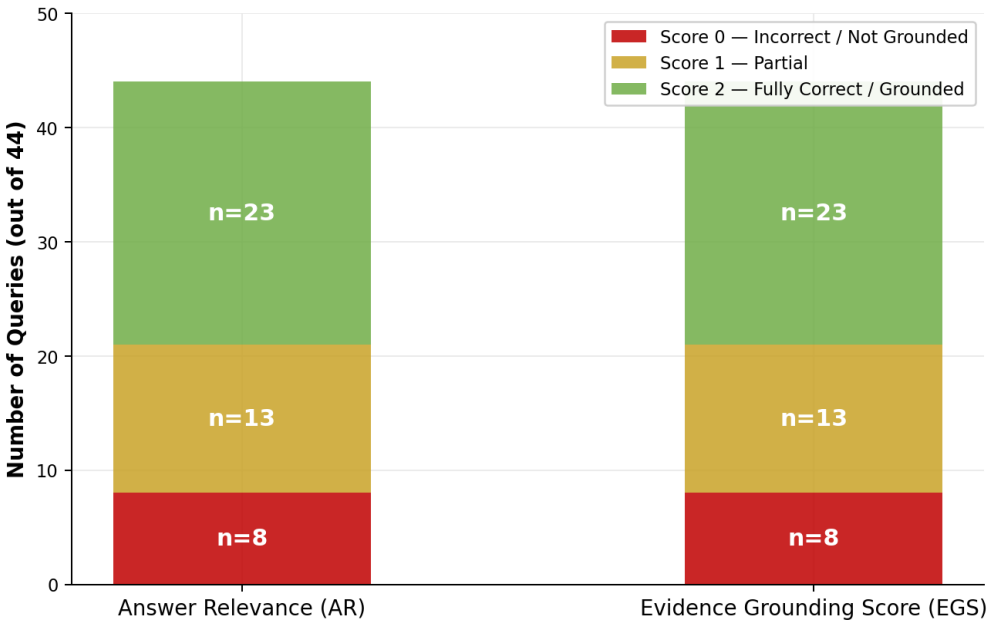

Figure 8.7: Score Distribution Across All Evaluation Queries

This stacked bar chart shows the distribution of raw evaluation scores (0, 1, and 2) across the evaluation queries for both AR and EGS. The distribution confirms that the majority of SummariV's outputs are either fully correct or partially correct, with very few complete failures. For AR, 23 queries received the maximum score of 2 (fully correct), 13 received score 1 (partially correct), and 8 received score 0 (incorrect). The EGS distribution is identical, reflecting the tight coupling between answer quality and evidence grounding — when the system fails to retrieve relevant evidence, the generated answer quality degrades proportionally. All six queries scoring 0 on AR also score 0 on EGS, confirming that retrieval failure is the primary bottleneck rather than language model generation. Score 0 cases are concentrated on abstract concept queries (paradigm shift, binary representation, foot drop, artificial intelligence) where visual content is less informative than spoken content, revealing a key limitation of the BLIP captioning model in purely abstract educational settings.

Overall, the results demonstrate that the proposed SummariV system effectively combines speech, vision, and retrieval mechanisms to deliver accurate, grounded, and temporally precise answers. The consistent improvement across configurations, strong baseline comparisons, and robust ablation results validate the effectiveness of the multimodal retrieval-augmented architecture. The system achieves high reliability across multiple evaluation dimensions, making it suitable for real-world educational video understanding applications.

CHAPTER – 9
CONCLUSION AND FUTURE WORK

9.1 Conclusion

In conclusion, the Multimodal Video Understanding and Question Answering System has demonstrated effectiveness in extracting and analyzing both audio and visual information from long-form video content. With the increasing availability of educational and informational videos, there is a growing need for intelligent systems that can efficiently retrieve relevant information and provide concise answers. The proposed system addresses this challenge by integrating speech recognition, image captioning, and semantic retrieval techniques into a unified framework.

The system enhances information accessibility by converting video content into structured textual and visual representations. By utilizing embedding techniques and similarity-based search, it enables accurate retrieval of relevant segments based on user queries. The integration of a language model ensures generation of meaningful and context-aware responses, while the extraction of corresponding video clips improves interpretability and user experience. Overall, the proposed framework provides an efficient, scalable, and intelligent solution for multimodal video analysis and question answering.

9.2 Future Work

In future work, the system can be enhanced by incorporating more advanced deep learning models to improve transcription accuracy, visual understanding, and answer generation quality. Expanding the system to handle multiple languages and diverse video domains can further increase its applicability and usability across global users. Additionally, optimizing the system for real-time processing can enable faster response generation for large-scale video datasets.

Further improvements may include integrating the framework into web-based or mobile applications to provide interactive user interfaces and wider accessibility. The incorporation of advanced multimodal fusion techniques and contextual reasoning models can enhance retrieval accuracy and response relevance. Future research can also explore personalization features, enabling the system to adapt to user preferences and learning patterns, thereby creating a more intelligent and user-centric video understanding platform.

REFERENCES & BIBLIOGRAPHY

[1]. A. Karpathy and L. Fei-Fei, “Deep visual-semantic alignments for generating image descriptions,” in Proc. IEEE CVPR, 2015.

[2]. L. Zhou et al., “Deep learning for video summarization: A survey,” IEEE Transactions on Pattern Analysis and Machine Intelligence, 2018.

[3]. K. Zhang et al., “Video summarization with long short-term memory,” IEEE Transactions on Multimedia, 2021.

[4]. T. Brown et al., “Language models are few-shot learners,” in Advances in Neural Information Processing Systems (NeurIPS), 2020.

[5]. A. Radford et al., “Language models are unsupervised multitask learn ers,” OpenAI, Tech. Rep., 2019.

[6]. Z. Chen et al., “Reducing hallucinations in large language models via grounding,” in Proc. ACL, 2023.

[7]. J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in Proc. NAACL-HLT, 2019.

[8]. Y. Lin et al., “BERT for video language understanding,” in Proc. IEEE/CVF ICCV, 2019.

[9]. A. Baevski et al., “wav2vec 2.0: A framework for self-supervised learning of speech representations,” in Proc. NeurIPS, 2020.

[10]. P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in Proc. NeurIPS, 2020.

[11]. A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in Proc. ICML, 2023.

[12]. J. Li, D. Li, C. Xiong, and S. Hoi, "BLIP: Bootstrapping language-image pre-training for unified vision-language understanding and generation," in Proc. ICML, 2022.

[13]. N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese

BERT-Networks,” in Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2019

[14]. J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," IEEE Trans. Big Data, vol. 7, no. 3, pp. 535–547, 2021.

[15]. C. Raffel et al., “Exploring the limits of transfer learning with a unified text-to-text transformer,” Journal of Machine Learning Research, 2020.

[16]. A. Radford et al., “Learning transferable visual models from natural language supervision,” in Proc. ICML, 2021.

[17]. Y. Zhang et al., “EDU-VSUM: A dataset for educational video summa rization,” arXiv preprint arXiv:2303.07284, 2023

[18]. Y. Jang et al., “Video question answering with spatio-temporal reason ing,” in Proc. AAAI, 2020.

[19]. H. Tan and M. Bansal, “Multimodal transformer for video retrieval,” in Proc. ECCV, 2020.

[20]. X. Li et al., “VideoLLM: Modeling video sequence with large language models,” arXiv preprint arXiv:2209.12356, 2022.

APPENDIX A

FULL CODE

BACK END

GPU CHECK & INSTALL DEPENDENCIES

```
# Check GPU availability (useful for faster model execution)
!nvidia-smi

# Install required libraries for the project
!pip install -q openai-whisper              # Speech-to-text (ASR)
!pip install -q sentence-transformers       # Embedding generation
!pip install -q faiss-cpu                   # Vector database
!pip install -q ffmpeg-python               # Video processing
!pip install -q torch torchvision torchaudio
!pip install -q transformers timm pillow opencv-python sentencepiece

# Install ffmpeg system package
!apt-get update
!apt-get install -y ffmpeg
```

CREATE PROJECT FOLDERS

```
import os

# Create necessary folders to organize project files
os.makedirs("video", exist_ok=True)        # Stores input video
os.makedirs("frames", exist_ok=True)       # Stores extracted frames
os.makedirs("transcripts", exist_ok=True)  # Stores text data
os.makedirs("embeddings", exist_ok=True)   # Stores FAISS indexes
os.makedirs("clips", exist_ok=True)        # Stores output clips

print("Folders created successfully")
```

UPLOAD VIDEO

```
from google.colab import files

# Upload video file from local system
uploaded = files.upload()

# Rename uploaded file to standard name
for name in uploaded:
    os.rename(name, "video/input.mp4")
```

```
print("Video saved as video/input.mp4")
```

SPEECH-TO-TEXT

```
import whisper
import json

# Load Whisper model (base version for balance of speed and accuracy)
model = whisper.load_model("base")

# Transcribe video audio with timestamps
result = model.transcribe(
    "video/input.mp4",
    word_timestamps=True
)

# Store segments with text + timestamps
segments = []
for seg in result["segments"]:
    segments.append({
        "text": seg["text"].strip(),
        "start": seg["start"],
        "end": seg["end"]
    })

# Save transcription to JSON file
with open("transcripts/transcript.json", "w") as f:
    json.dump(segments, f, indent=2)

print("Transcription completed")
print("Total segments:", len(segments))
```

TEXT EMBEDDINGS + FAISS INDEX

```
import faiss
import numpy as np
from sentence_transformers import SentenceTransformer

# Load sentence transformer model
embed_model = SentenceTransformer("all-MiniLM-L6-v2")

# Extract text data from segments
texts = [seg["text"] for seg in segments]

# Convert text into embeddings (vector representation)
text_embeddings = embed_model.encode(texts, convert_to_numpy=True)
```

```
# Create FAISS index
dim = text_embeddings.shape[1]
text_index = faiss.IndexFlatL2(dim)
text_index.add(text_embeddings)

# Save index
faiss.write_index(text_index, "embeddings/text.index")

print("Text FAISS index created")
```

FRAME EXTRACTION

```
import cv2

# Open video file
cap = cv2.VideoCapture("video/input.mp4")

# Get frames per second
fps = cap.get(cv2.CAP_PROP_FPS)

# Extract one frame every 2 seconds
frame_interval = int(fps * 2)
frame_id = 0
saved = 0

# Loop through video frames
while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    # Save frame at defined interval
    if frame_id % frame_interval == 0:
        timestamp = frame_id / fps
        cv2.imwrite(f"frames/frame_{timestamp:.2f}.jpg", frame)
        saved += 1

    frame_id += 1

cap.release()
print(f"Extracted {saved} frames")
```

IMAGE CAPTIONING

```
from transformers import BlipProcessor, BlipForConditionalGeneration
from PIL import Image
import torch
```

```python
import os
import json

# Use GPU if available
device = "cuda" if torch.cuda.is_available() else "cpu"

# Load BLIP captioning model
processor = BlipProcessor.from_pretrained("Salesforce/blip-image-captioning-
base")
blip_model = BlipForConditionalGeneration.from_pretrained(
    "Salesforce/blip-image-captioning-base"
).to(device)
visual_segments = []

# Generate captions for each frame
for img in sorted(os.listdir("frames")):
    image = Image.open(f"frames/{img}").convert("RGB")
    inputs = processor(image, return_tensors="pt").to(device)
    out = blip_model.generate(**inputs, max_new_tokens=30)
    caption = processor.decode(out[0], skip_special_tokens=True)
    timestamp = float(img.split("_")[1].replace(".jpg", ""))
    visual_segments.append({
        "text": caption,
        "start": timestamp,
        "end": timestamp + 1
    })

# Save captions
with open("transcripts/visual_segments.json", "w") as f:
    json.dump(visual_segments, f, indent=2)
print("Visual captions generated:", len(visual_segments))
```

VISUAL EMBEDDINGS + FAISS

```python
# Extract captions
visual_texts = [v["text"] for v in visual_segments]

# Convert captions into embeddings
visual_embeddings = embed_model.encode(visual_texts, convert_to_numpy=True)

# Create FAISS index
visual_index = faiss.IndexFlatL2(dim)
visual_index.add(visual_embeddings)

# Save index
faiss.write_index(visual_index, "embeddings/visual.index")
print("Visual FAISS index created")
```

COMBINED MULTIMODAL DATABASE

```
combined_db = []

# Add audio/text data
for i in range(len(segments)):
    combined_db.append({
        "modality": "Audio",
        "text": segments[i]["text"],
        "start": segments[i]["start"],
        "end": segments[i]["end"],
        "embedding": text_embeddings[i]
    })

# Add visual data
for i in range(len(visual_segments)):
    combined_db.append({
        "modality": "Visual",
        "text": visual_segments[i]["text"],
        "start": visual_segments[i]["start"],
        "end": visual_segments[i]["end"],
        "embedding": visual_embeddings[i]
    })
print("Total entries in combined DB:", len(combined_db))
```

LOAD LLM

```
from transformers import pipeline

# Load lightweight instruction-based model
qa_pipeline = pipeline(
    "text2text-generation",
    model="google/flan-t5-base",
    max_length=64
)
print("LLM loaded successfully")
```

FULL VIDEO SUMMARY

```
# Function to split long text into chunks
def chunk_text(text, chunk_size=500):
    words = text.split()
    for i in range(0, len(words), chunk_size):
        yield " ".join(words[i:i + chunk_size])

# Combine all speech text
full_speech = " ".join([seg["text"] for seg in segments])
```

```python
# Split into chunks
speech_chunks = list(chunk_text(full_speech, 400))
chunk_summaries = []

# Summarize each chunk
for chunk in speech_chunks:
    prompt = f"""
Summarize the following lecture content simply.
Text:
{chunk}
"""
    summary = qa_pipeline(prompt)[0]["generated_text"]
    chunk_summaries.append(summary)

# Final summary
full_summary = "\n\n".join(chunk_summaries)
print(full_summary)
```

QUERY PROCESSING + RETRIEVAL + ANSWER

```python
# Take user query
query = input("Ask your question: ")

# Convert query to embedding
query_emb = embed_model.encode([query], convert_to_numpy=True)

# Retrieve top text segments
D_t, I_t = text_index.search(query_emb, k=2)
text_context = " ".join(segments[i]["text"] for i in I_t[0])

# Retrieve top visual segments
D_v, I_v = visual_index.search(query_emb, k=2)
visual_context = " ".join(visual_segments[i]["text"] for i in I_v[0])

# Combine context
combined_context = text_context + " " + visual_context

# Generate answer
prompt = f"""
Answer ONLY using the context.
Be very short.
Context:
{combined_context}
Question:
{query}
"""
answer = qa_pipeline(prompt)[0]["generated_text"]
print("Final Answer:", answer)
```

VIDEO CLIP EXTRACTION

```
import subprocess
from IPython.display import Video, display

# Select best matching segment
best_segment = segments[I_t[0][0]]

# Define clip time range
start = max(0, best_segment["start"] - 0.9)
end = best_segment["end"] + 0.5

# Extract clip using ffmpeg
subprocess.run([
    "ffmpeg", "-y",
    "-i", "video/input.mp4",
    "-ss", str(start),
    "-to", str(end),
    "clips/answer_clip.mp4"
])

# Display video clip
display(Video("clips/answer_clip.mp4", embed=True))
```

BACKEND-FRONTEND INTEGRATION USING FASTAPI

```
from fastapi import FastAPI, File, UploadFile, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import FileResponse
from pydantic import BaseModel

# ── App setup
app = FastAPI(title="SummariV API")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)

WORK_DIR = Path("sessions")
WORK_DIR.mkdir(exist_ok=True)

# ── Load models once at startup
print("Loading models…")
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
```

```
whisper_model   = whisper.load_model("base")
embed_model     = SentenceTransformer("all-MiniLM-L6-v2")
blip_processor  = BlipProcessor.from_pretrained("Salesforce/blip-image-
captioning-base")
blip_model      = BlipForConditionalGeneration.from_pretrained(
                      "Salesforce/blip-image-captioning-base").to(DEVICE)
qa_pipeline     = hf_pipeline("text2text-generation", model="google/flan-t5-
base",
                                max_length=128, device=0 if DEVICE=="cuda"
else -1)
print("Models ready.")


# ── Session helpers
def session_dir(vid: str) -> Path:
    p = WORK_DIR / vid
    p.mkdir(parents=True, exist_ok=True)
    return p


# ── /upload
@app.post("/upload")
async def upload_video(video: UploadFile = File(...)):
    vid = str(uuid.uuid4())[:8]
    sess = session_dir(vid)

    # 1. Save video
    video_path = sess / "input.mp4"
    with open(video_path, "wb") as f:
        f.write(await video.read())

    # 2. Transcribe
    result = whisper_model.transcribe(str(video_path), word_timestamps=True)
    segments = [{"text": s["text"].strip(), "start": s["start"], "end":
s["end"]}
                for s in result["segments"]]
    (sess / "segments.json").write_text(json.dumps(segments))

    # 3. Extract frames (1 per 2 s)
    frames_dir = sess / "frames"
    frames_dir.mkdir(exist_ok=True)
    cap = cv2.VideoCapture(str(video_path))
    fps = cap.get(cv2.CAP_PROP_FPS) or 25
    interval = int(fps * 2)
    fid = saved = 0
    while cap.isOpened():
        ret, frame = cap.read()
        if not ret: break
        if fid % interval == 0:
```

```
            ts = fid / fps
            cv2.imwrite(str(frames_dir / f"frame_{ts:.2f}.jpg"), frame)
            saved += 1
        fid += 1
    cap.release()

    # 4. BLIP captions
    visual_segments = []
    for img_name in sorted(os.listdir(frames_dir)):
        img = Image.open(frames_dir / img_name).convert("RGB")
        inputs = blip_processor(img, return_tensors="pt").to(DEVICE)
        out = blip_model.generate(**inputs, max_new_tokens=30)
        caption = blip_processor.decode(out[0], skip_special_tokens=True)
        ts = float(img_name.split("_")[1].replace(".jpg",""))
        visual_segments.append({"text": caption, "start": ts, "end": ts+1})
    (sess / "visual_segments.json").write_text(json.dumps(visual_segments))

    # 5. Build FAISS indexes
    texts        = [s["text"] for s in segments]
    t_embs       = embed_model.encode(texts, convert_to_numpy=True)
    text_index   = faiss.IndexFlatL2(t_embs.shape[1])
    text_index.add(t_embs)
    faiss.write_index(text_index, str(sess / "text.index"))
    np.save(str(sess / "text_embs.npy"), t_embs)

    vis_texts    = [v["text"] for v in visual_segments]
    v_embs       = embed_model.encode(vis_texts, convert_to_numpy=True)
    vis_index    = faiss.IndexFlatL2(v_embs.shape[1])
    vis_index.add(v_embs)
    faiss.write_index(vis_index, str(sess / "visual.index"))

    # 6. Quick summary (chunked)
    full_speech = " ".join(texts)
    words = full_speech.split()
    chunks = [" ".join(words[i:i+400]) for i in range(0, len(words), 400)]
    chunk_summaries = []
    for chunk in chunks[:4]:   # cap at 4 chunks for speed
        prompt = f"Summarize this educational lecture excerpt:\n\n{chunk}"
        chunk_summaries.append(qa_pipeline(prompt)[0]["generated_text"])
    summary = " ".join(chunk_summaries)

    return {"video_id": vid, "summary": summary,
            "segments": len(segments), "frames": saved}


# — /query
class QueryRequest(BaseModel):
    video_id: str
    query: str
```

```
@app.post("/query")
async def query_video(req: QueryRequest):
    sess = session_dir(req.video_id)
    if not (sess / "text.index").exists():
        raise HTTPException(404, "Session not found — upload video first")

    segments        = json.loads((sess / "segments.json").read_text())
    visual_segments = json.loads((sess /
"visual_segments.json").read_text())
    text_index      = faiss.read_index(str(sess / "text.index"))
    vis_index       = faiss.read_index(str(sess / "visual.index"))

    q_emb = embed_model.encode([req.query], convert_to_numpy=True)

    # Retrieve top-2 from each modality
    _, I_t = text_index.search(q_emb, k=2)
    _, I_v = vis_index.search(q_emb, k=2)

    text_ctx   = " ".join(segments[i]["text"]        for i in I_t[0])
    visual_ctx = " ".join(visual_segments[i]["text"] for i in I_v[0])
    combined   = text_ctx + " " + visual_ctx

    prompt = (f"Answer ONLY using the context below. Be concise (1–2
sentences).\n\n"
              f"Context: {combined}\n\nQuestion: {req.query}")
    answer = qa_pipeline(prompt)[0]["generated_text"]

    # Best segment for clip
    best     = segments[I_t[0][0]]
    clip_start = max(0, best["start"] - 0.9)
    clip_end   = best["end"] + 0.5

    # Extract clip with ffmpeg
    clip_path = sess / "answer_clip.mp4"
    subprocess.run([
        "ffmpeg", "-y", "-i", str(sess / "input.mp4"),
        "-ss", str(clip_start), "-to", str(clip_end),
        str(clip_path)
    ], capture_output=True)

    # Simple Answer Relevance score (cosine similarity)
    a_emb  = embed_model.encode([answer],    convert_to_numpy=True)
    q_emb2 = embed_model.encode([req.query], convert_to_numpy=True)
    cos    = float(np.dot(a_emb[0], q_emb2[0]) /
                   (np.linalg.norm(a_emb[0]) * np.linalg.norm(q_emb2[0]) +
1e-9))
```

```
        return {
            "answer":           answer,
            "answer_relevance": round(cos, 3),
            "text_match":       best["text"],
            "visual_match":     visual_segments[I_v[0][0]]["text"],
            "clip_start":       clip_start,
            "clip_end":         clip_end,
        }


    # ── /clip/{video_id}
    @app.get("/clip/{video_id}")
    def get_clip(video_id: str):
        clip = WORK_DIR / video_id / "answer_clip.mp4"
        if not clip.exists():
            raise HTTPException(404, "Clip not found")
        return FileResponse(str(clip), media_type="video/mp4")

    # ── Health check
    @app.get("/")
    def health():
        return {"status": "ok", "device": DEVICE}
```

REMOTE ACCESS WITH NGROK

```
import nest_asyncio, asyncio, os, time, subprocess, re, json, urllib.request
nest_asyncio.apply()

NGROK_TOKEN = ""  # ← paste your token here

if NGROK_TOKEN:
    ngrok.set_auth_token(NGROK_TOKEN)
else:
    print(" No ngrok token set — tunnel may be rate-limited or fail.")
    print("Get a free token at https://dashboard.ngrok.com/get-started/your-
authtoken")

# Start ngrok
ngrok_proc = subprocess.Popen(
    ["ngrok", "http", "8000", "--log", "stdout", "--log-format", "json"],
    stdout=subprocess.PIPE, stderr=subprocess.STDOUT, text=True,
)

def _run_server():
    uvicorn.run(app, host="0.0.0.0", port=8000, log_level="warning")


# Start uvicorn in a background thread
server_thread = threading.Thread(target=_run_server, daemon=True)
```

```python
server_thread.start()

# Open ngrok tunnel
public_url = ngrok.connect(8000).public_url

print("=" * 60)
print(f" API BASE URL : {public_url}")
print(f" Swagger docs : {public_url}/docs")
print(f" Status check : {public_url}/status")
print("=" * 60)
print("Paste the API BASE URL into the frontend settings bar.")
print("Pipeline ready :", APP_STATE["ready"])
if APP_STATE["ready"]:
    print(f"  Audio segments : {len(APP_STATE['segments'])}")
    print(f"  Visual caps    : {len(APP_STATE['visual_segments'])}")
```

FRONT END

```html
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>SummariV — Video Intelligence Platform</title>
  <style>
    *, *::before, *::after {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
    }

    :root {
      --bg: #f5f5f3;
      --surface: #ffffff;
      --border: #e0dfd8;
      --text: #1a1a18;
      --muted: #6b6b66;
      --blue: #185fa5;
      --blue-light: #e6f1fb;
      --blue-mid: #378add;
      --green: #3b6d11;
      --green-light: #eaf3de;
      --red: #a32d2d;
      --red-light: #fcebeb;
    }
    body {
      font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
      background: var(--bg);
```

```
  color: var(--text);
  min-height: 100vh;
  padding: 2rem 1rem;
}

.container {
  max-width: 700px;
  margin: 0 auto;
}

header {
  text-align: center;
  margin-bottom: 2.5rem;
}

header h1 {
  font-size: 28px;
  font-weight: 600;
}

header p {
  color: var(--muted);
  font-size: 14px;
  margin-top: 6px;
}

.card {
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 12px;
  padding: 1.5rem;
  margin-bottom: 1.25rem;
}

.card-label {
  font-size: 11px;
  font-weight: 600;
  color: var(--muted);
  text-transform: uppercase;
  margin-bottom: 14px;
}

/* Upload */
.drop-zone {
  border: 2px dashed var(--border);
  border-radius: 10px;
  padding: 2.5rem 1rem;
  text-align: center;
```

```
  cursor: pointer;
}

.drop-zone:hover {
  border-color: var(--blue-mid);
  background: var(--blue-light);
}

#file-input {
  display: none;
}

/* Query */
.query-row {
  display: flex;
  gap: 10px;
}

.query-row input {
  flex: 1;
  padding: 11px 14px;
  border: 1px solid var(--border);
  border-radius: 8px;
}

.ask-btn {
  padding: 11px 22px;
  background: var(--blue);
  color: white;
  border: none;
  border-radius: 8px;
  cursor: pointer;
}

.ask-btn:disabled {
  background: var(--border);
  cursor: not-allowed;
}

/* Answer */
#answer-panel {
  display: none;
}

.answer-box {
  background: #f9f9f7;
  border-radius: 8px;
  padding: 1rem;
```

```
      margin-bottom: 14px;
    }
  </style>
</head>

<body>
  <div class="container">
    <header>
      <h1>SummariV</h1>
      <p>Multimodal video question answering for educational content</p>
    </header>

    <!-- Upload -->
    <div class="card">
      <div class="card-label">Step 1 — Upload Video</div>

      <div class="drop-zone" onclick="document.getElementById('file-
input').click()">
        Click or Drag & Drop Video
      </div>

      <input type="file" id="file-input" accept="video/*"
onchange="handleFile(this.files[0])" />
    </div>

    <!-- Query -->
    <div class="card">
      <div class="card-label">Step 2 — Ask a Question</div>
      <div class="query-row">
        <input type="text" id="query-input" placeholder="Ask a question..." />
        <button class="ask-btn" id="ask-btn" onclick="submitQuery()" disabled>
          Ask
        </button>
      </div>
    </div>

    <!-- Answer -->
    <div class="card" id="answer-panel">
      <div class="card-label">Answer</div>
      <div class="answer-box" id="answer-text"></div>
    </div>
  </div>

  <script>
    const API_BASE = "http://localhost:8000";
    let videoId = null;

    async function handleFile(file) {
```

```
      if (!file) return;

      const form = new FormData();
      form.append("video", file);

      try {
        const res = await fetch(API_BASE + "/upload", {
          method: "POST",
          body: form,
        });

        const data = await res.json();
        videoId = data.video_id;

        document.getElementById("ask-btn").disabled = false;
      } catch (err) {
        alert("Upload failed");
      }
    }

    async function submitQuery() {
      const q = document.getElementById("query-input").value;

      const res = await fetch(API_BASE + "/query", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ video_id: videoId, query: q }),
      });

      const data = await res.json();

      document.getElementById("answer-text").textContent = data.answer;
      document.getElementById("answer-panel").style.display = "block";
    }
  </script>
</body>

</html>
```

SCREENSHOTS

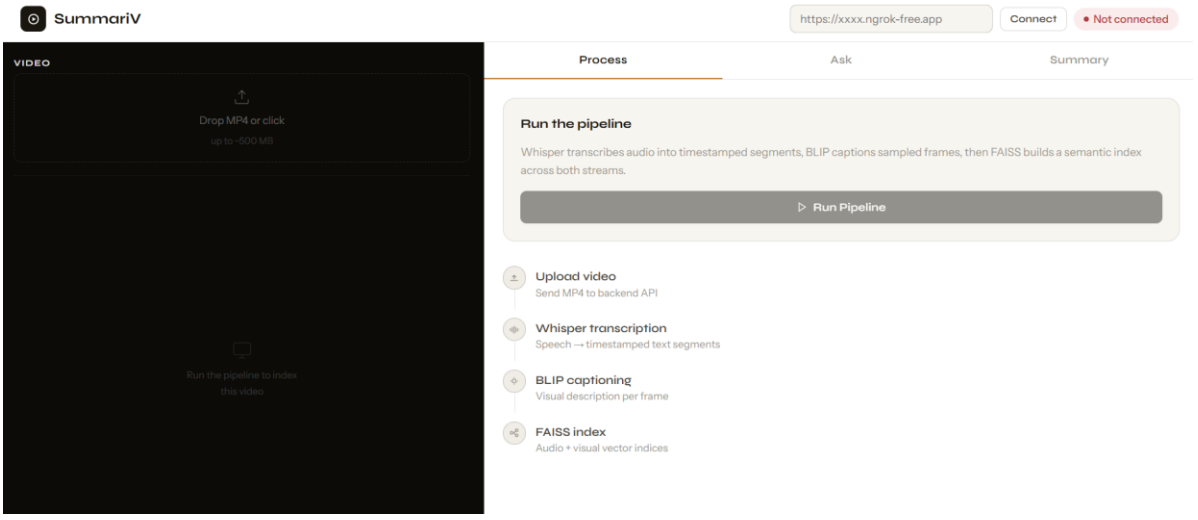

Figure A.1: Video upload and pipeline interface

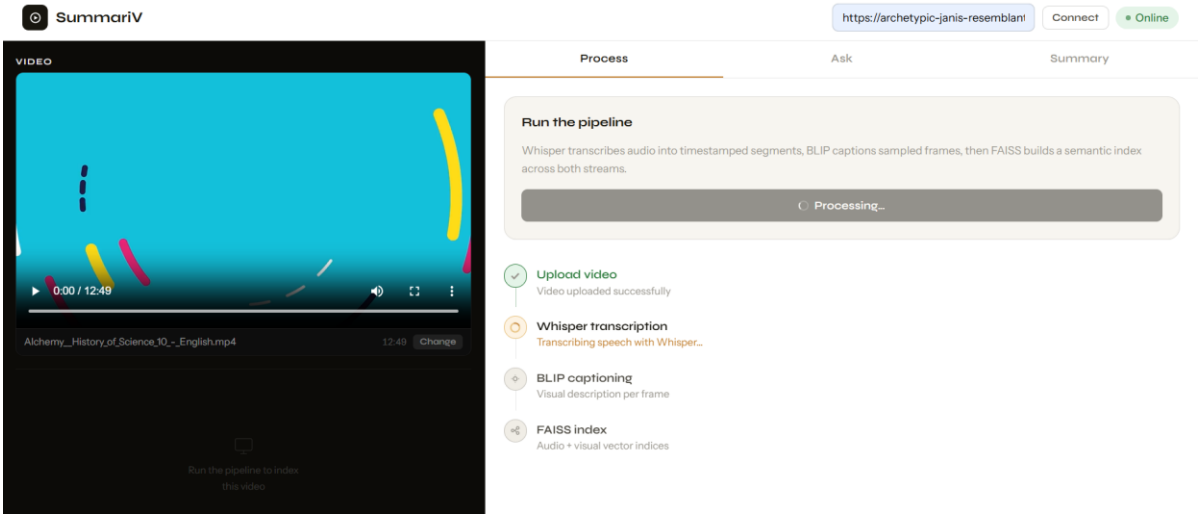

Figure A.2: Pipeline processing stages

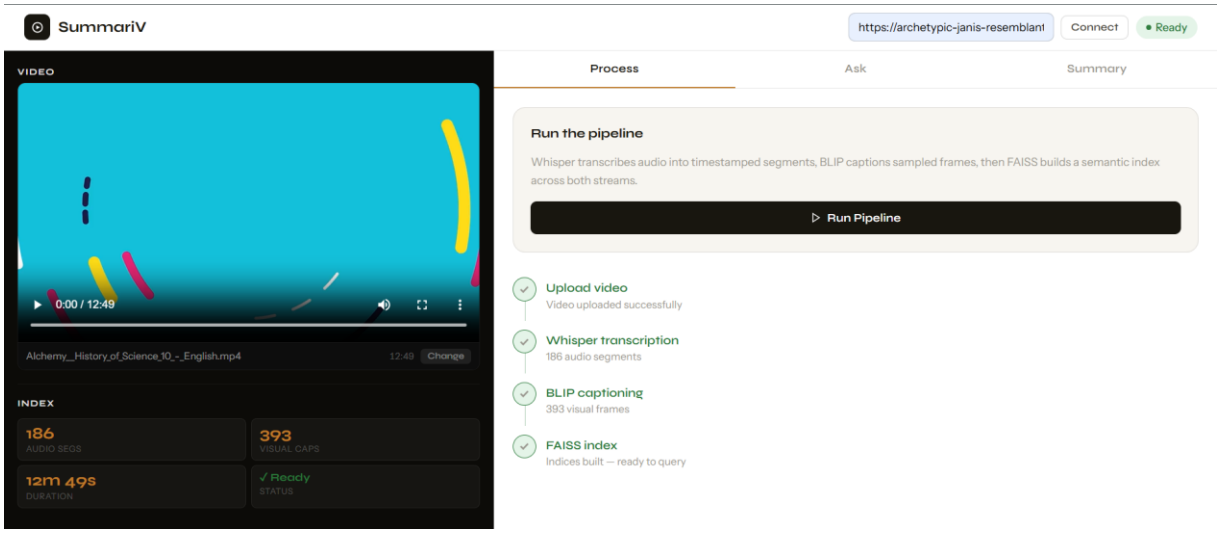

Figure A.3: Completed indexing and system ready state

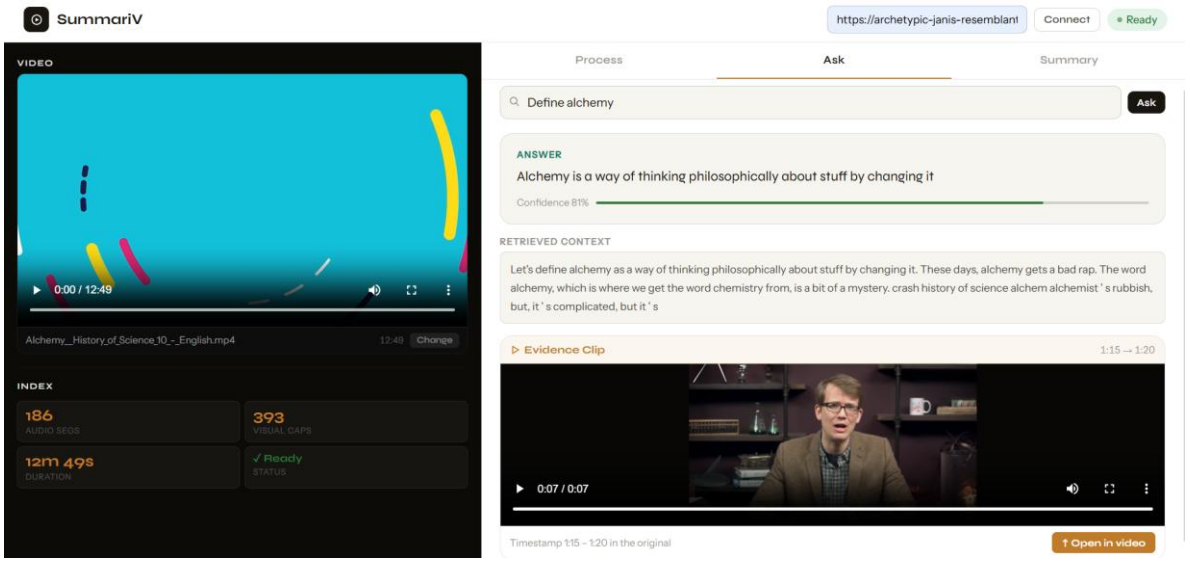

Figure A.4: Evidence-based answer with video clip

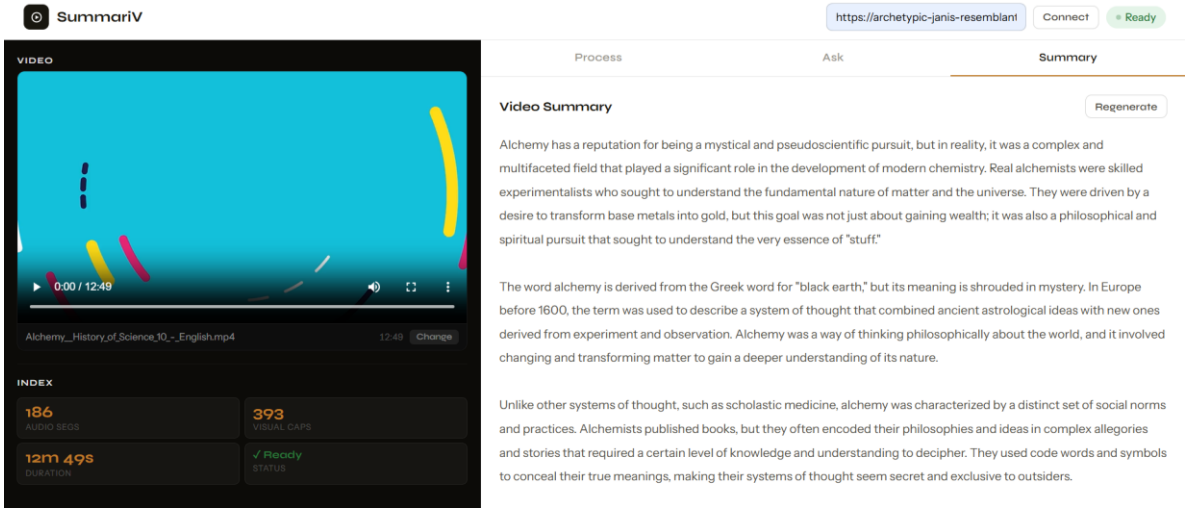

Figure A.5: Overall video summary output

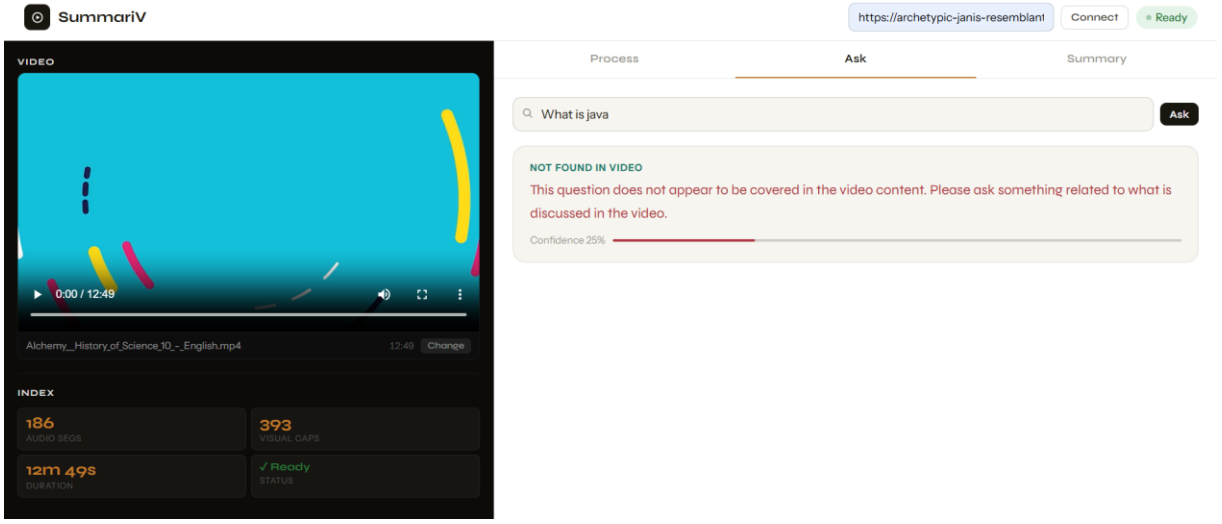

Figure A.6: Query handling with low confidence response

APPENDIX – B

Mapping of Sustainable Development Goals (SDGs)

The SummariV a Multimodal Retrieval-Augmented Generation framework for evidence-grounded video question answering on educational content aligns with several United Nations Sustainable Development Goals (SDGs). By combining automated speech recognition, visual frame captioning, semantic embedding retrieval, and neural answer generation, the system improves access to accurate knowledge, enhances intelligent information retrieval infrastructure, and supports transparent, evidence-based decision-making.

Table B.1: Mapping of SDGs

Sustainable Development Goals (SDGs)	Observation
SDG 4: Quality Education	SummariV supports inclusive learning by enabling users to query educational videos and retrieve precise, evidence-based answers. It promotes self-paced and personalized learning by reducing the need to navigate entire lectures.
SDG 9: Industry, Innovation, and Infrastructure	The system integrates advanced AI components such as Whisper, BLIP, SBERT, FAISS, and Flan-T5 to build a scalable multimodal RAG pipeline. It demonstrates innovation in intelligent information retrieval for real-world educational use.

By addressing SDG 4 (Quality Education) and SDG 9 (Industry, Innovation, and Infrastructure), this project contributes to the development of intelligent, accessible, and trustworthy educational technologies. The SummariV framework highlights the potential of multimodal, evidence-grounded AI to improve learning accessibility, ensure factual and reliable information delivery, and support collaborative innovation. It promotes a future where AI-driven systems enhance self-directed learning, maintain transparency, and provide equitable access to quality educational resources for diverse learners.

Signature of the Guide